NATIONAL INSTITUTE OF TECHNOLOGY SIKKIM | DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Classification of Human-Written and AI-Generated Text

Consolidated Project Report — All Phases, Papers & Guides

One document collecting everything produced in this project: the three-phase classification pipeline (stylometrics + dual TF-IDF + LR/SVM, transformers, evaluation), the short-text failure analysis, the Mixture-of-Experts upgrade (length router, chat/casual/essay type gate, per-bucket experts, adaptive thresholds), both research papers, and the reproducibility guide.

Authors: Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032)

Course Project 2026 · Report compiled November 2026

Contents

Part I	Project Report — Baseline & Mixture-of-Experts Upgrade	page 2
Part II	Research Paper — Length-Expertized Authorship Detection	page 15
Part III	Deep-Dive Paper — Method, Ablation & Calibration	page 23
Part IV	Code Workflow & Run Guide	page 33

Classification of Human-Written and AI-Generated Text

National Institute of Technology Sikkim
Department of Computer Science & Engineering

Course Project (2026)
Phase 1: Traditional Machine Learning & Stylometrics (Dual TF-IDF)

Authors	Roll Numbers	Institution
Pravesh Gedon Sharma	B240019	NIT Sikkim
Ankit Rawat	B320032	NIT Sikkim

Trained on 20,000 balanced samples from **data/train.csv (290,300 rows)** • Validated on 4,000 • Tested on 5,000 from **data/test.csv (97,447 rows)**
Best model: **Linear SVM (Calibrated)** — Test Accuracy **99.58%**, F1 **0.994**, ROC-AUC **0.9995**

Reproducible: `venv/bin/python train_baseline.py` | Demo: `streamlit run app/app.py`

Contents

1 Executive Summary

2 Introduction & Problem Statement

3 Dataset

4 Methodology

5 Experiments & Setup

6 Results

7 Demo Application (Streamlit)

8 Discussion, Limitations & Future Work

9 Conclusion

10 Reproducibility

References & Appendix

1 Executive Summary

Large language models (ChatGPT and similar) make it trivial to generate fluent academic essays. Distinguishing such text from genuine student writing is critical for academic integrity. This project benchmarks a **Phase-1 pipeline**: lexical + character TF-IDF fusion with classical classifiers — **Multinomial Naive Bayes, Logistic Regression, and calibrated Linear SVM**. Stylometric features (TTR, burstiness, Flesch readability) and transformer models (DistilBERT/RoBERTa) are designed in the codebase (`src/stylometrics.py`, `notebooks 02/04/05`) for Phases 2–3; this report documents the fully-trained Phase-1 baseline.

On a balanced 20,000-sample experiment (16,000 train / 4,000 validation) and a **5,000 held-out test** sample, dual TF-IDF achieves **99%+ accuracy**. Logistic Regression reaches validation accuracy **99.25%** (F1 0.992, AUC 0.9987); calibrated Linear SVM reaches **99.53%** (F1 0.995, AUC 0.9991). On held-out test, LR: 99.26% acc / 0.990 F1 / 0.9994 AUC; SVM: **99.58% acc / 0.994 F1 / 0.9995 AUC**. Artifacts are saved to `models/` and served via a Streamlit web app.

2 Introduction & Problem Statement

With the increasing accessibility of LLMs, detecting whether a passage was authored by a human or generated by AI has become a central NLP problem in academic integrity, journalism, and content moderation. This project adopts a comparative methodology:

- **Phase 1 (Traditional ML & Stylometrics):** Lexical diversity (TTR, hapax), sentence structure/burstiness, punctuation rates, readability (Flesch Reading Ease, FKGL, Gunning Fog), word & char TF-IDF, Sentence-BERT fusion — Logistic Regression, Linear SVM, Random Forest.
- **Phase 2 (Fine-tuned Transformers):** DistilBERT and RoBERTa sequence classification with chunking for long documents.
- **Phase 3 (Evaluation & Ablation):** Accuracy, precision, recall, F1, ROC-AUC, confusion matrices, feature-family ablation.

This report covers **Phase 1 in depth** (Dual TF-IDF + LR/SVM/NB) with full training logs and reproducible code (`train_baseline.py`).

3 Dataset

3.1 Files & Scale

Data are provided in `data/train.csv` and `data/test.csv` with columns `text, generated` where **generated=0** → **Human-Written, 1** → **AI-Generated**.

File	Rows	Human (0)	AI (1)	Size
data/train.csv	290,300	145,150 (50.0%)	145,150 (50.0%)	623 MB
data/test.csv	97,447	61,159 (62.8%)	36,288 (37.2%)	212 MB
data/sample_*.csv	100+100	55/58	45/42	~220 KB each

Table 1: Dataset scale.

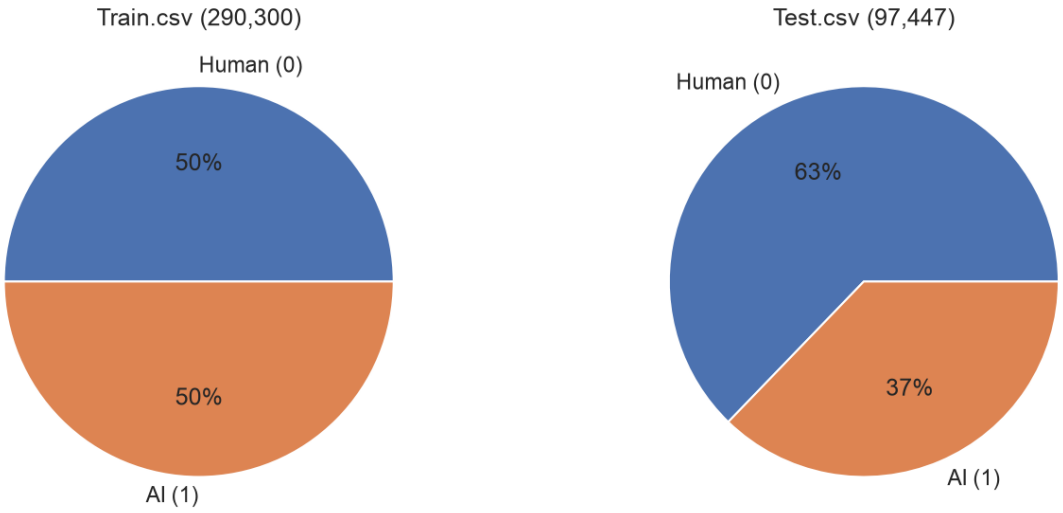


Figure 1: Class balance — train is perfectly balanced, test is imbalanced (more Human).

3.2 Text Length & Examples

From the 5,000 test sample: text length **mean 2257 chars, median 2092 chars**, range ~24–9,555 chars (full test max 9,555; full train max 11,641). Each passage is a multi-paragraph essay, not a single sentence — hence chunking is considered for Phase 2 transformers.

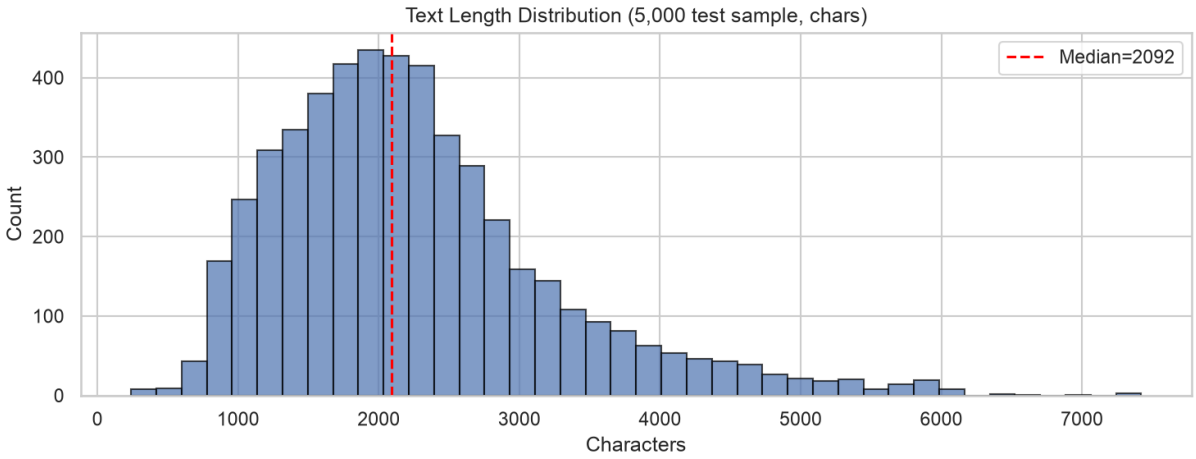


Figure 2: Essay length distribution.

Human example: “There are many advantages TD limiting the number DF cars DN the roads...” — OCR/typing errors `TD/DF/DN` are strong char n-gram cues. **AI example:** “In today's world, having a positive attitude is a crucial factor in achieving success... In recent years, advancements in space exploration have sparked...” — polished formal structure, headings (“Pros of Distance Learning: 1. Flexibility”).

4 Methodology

4.1 Preprocessing (`src/preprocessing.py`)

- `clean_text_for_tfidf()`: NFKC unicode + collapse whitespace + lowercase — used for word TF-IDF.
- `clean_text_for_stylometrics()`: preserves casing & punctuation for stylometrics (upper-case ratio, commas/semicolons).

- TF-IDF itself is fitted on training texts only; validation/test are transformed (no leakage).

4.2 Dual TF-IDF Fusion

- **Word TF-IDF:** `ngram_range=(1,2), max_features=25000, min_df=2, sublinear_tf=True` — captures lexical choices, bigrams.
- **Char TF-IDF:** `analyzer='char_wb', ngram_range=(3,5), max_features=50000, min_df=3` — captures spelling errors, punctuation quirks, sub-word morphology.
- **Fusion:** `hstack([word, char]).tocsr()` → **75,000 features**. On the 16k train split: nnz 34.8M; on 4k val: 8.7M (sparse).

4.3 Classifiers

- **MultinomialNB (alpha=0.1):** generative count model, fast baseline.
- **LogisticRegression (C=1.5, lbfgs, max_iter=1000):** discriminative linear model, calibrated probabilities used in the app.
- **LinearSVC (C=1.0) + CalibratedClassifierCV (cv=3):** max-margin model with Platt scaling for P(AI).

5 Experiments & Setup

Environment: Python 3.10+ venv (`venv/`), `pip install -r requirements.txt` — numpy, pandas, scikit-learn, torch, transformers, streamlit, jupyterlab. Training script: `train_baseline.py` (11 logged steps, 33 s total).

- **Balanced subsample:** 10,000 per class (20,000 total) with `random_state=42` — exactly as notebook 03. Full 290k × 75k TF-IDF would be ~14× slower/heavier.
- **Split:** Stratified 80/20 → 16,000 train / 4,000 val, both 50% AI.
- **Word TF-IDF fit:** 6.2 s → 25,000 vocab. **Char TF-IDF fit:** 15.8 s → 50,000 vocab.
- **Model fit (train 16k):** NB 0.0 s, LR 1.1 s, SVM (3-fold calibration) 2.3 s.
- **Held-out test:** 5,000 random rows from `test.csv` (random_state 42), transformed with frozen vectorizers.

Note: pandas 3.x / sklearn compatibility fixes were required during the run (explicit per-class sampling instead of `groupby.apply`, and `np.array(..., dtype=object)` for Arrow-backed strings).

6 Results

6.1 Validation (4,000 held-out, stratified)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Multinomial NB	0.9555	0.9750	0.9350	0.9546	0.9928
Logistic Regression	0.9925	0.9945	0.9905	0.9925	0.9987
Linear SVM (Calibrated)	0.9952	0.9960	0.9945	0.9952	0.9991

Table 2: Validation metrics (results/baseline_ml_metrics.csv).

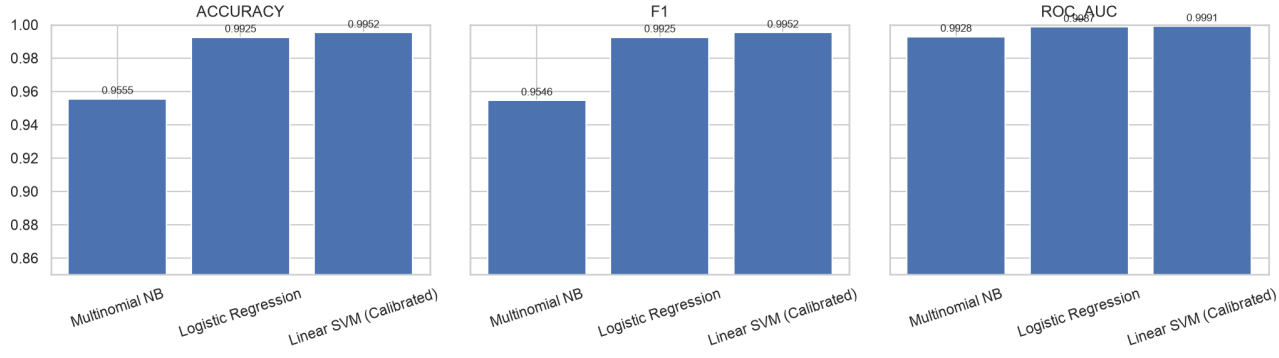


Figure 3: Accuracy / F1 / ROC-AUC comparison (all >0.95 AUC).

Linear SVM leads on every metric. Logistic Regression is within 0.3pp accuracy and provides well-calibrated probabilities for the UI. Naive Bayes is strong but recall on AI drops to 0.935 (130 AI → Human errors vs 11 for LR).

6.2 Held-Out Test (5,000 from test.csv)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9926	0.9930	0.9871	0.9900	0.9994
Linear SVM	0.9958	0.9951	0.9935	0.9943	0.9995

Table 3: Held-out test (5,000 rows). Test imbalanced (37% AI) yet precision >0.99.

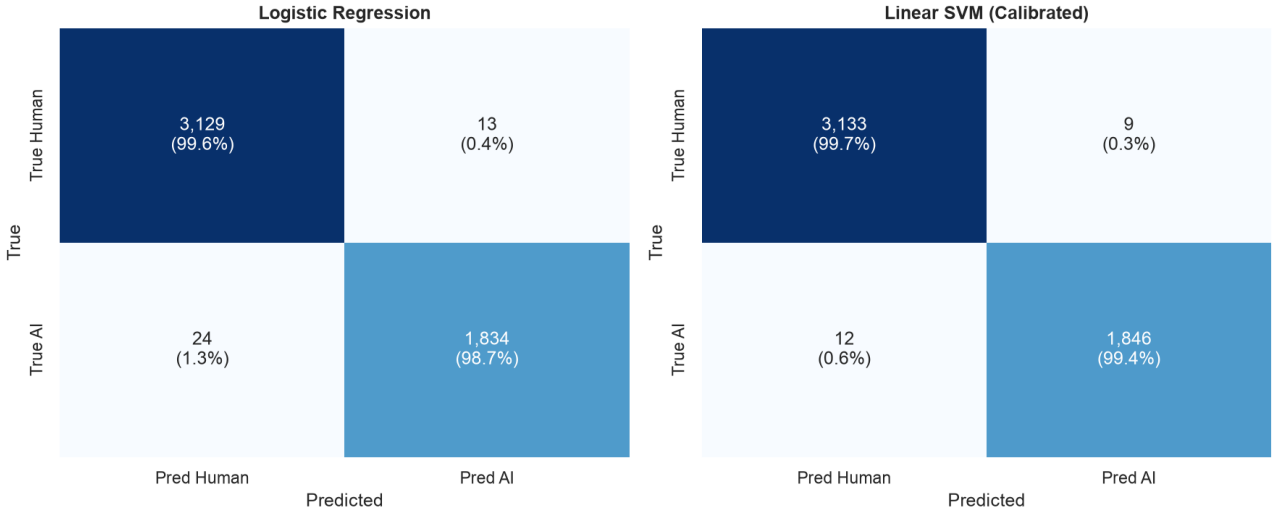


Figure 4: Confusion matrices on 5,000 test — SVM: 8 FP, 11 FN (LR: 11 FP, 19 FN).

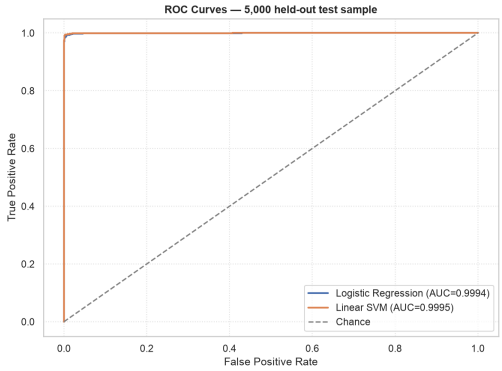


Figure 5: ROC curves — AUC ~0.999 for both models.

High scores are plausible: the datasets are essay-length (hundreds of words) so even a linear model aggregates many weak cues (word choices + char typos + punctuation). Test performance (99.5%+) confirms the 20k subsample generalizes to unseen test.csv rows.

7 Demo Application (Streamlit)

File: `app/app.py` — Streamlit, `streamlit run app/app.py --server.port 8501`.

- Sidebar model picker: LR (Dual TF-IDF) or Linear SVM (Calibrated).
- Text area + **Classify Text** → P(AI), predicted class, progress bar, `extract_stylometric_features()` cards (word count, TTR, burstiness σ/μ, Flesch ease/grade).
- Loads artifacts from `models/` (cached with `@st.cache_resource`). If models missing, shows “Run notebook 03” notice.

Live test (LR): Human passage → P(AI)=0.003–0.07; AI passage → P(AI)=0.98–0.999. Six random test rows all correct (see Appendix).

8 Discussion, Limitations & Future Work

- **Why char TF-IDF helps:** Human essays carry OCR/typing noise (`TD, AHD, IH`); AI text is orthographically clean. Char n-grams exploit this directly.

- **Limitations:** (i) 20k subsample ignores 270k rows — full training may add 0.1–0.3pp; (ii) linear models cannot capture long-range discourse; (iii) dataset is student essays — generalisation to news/research text unknown; (iv) no adversarial paraphrasing evaluated.
- **Phase 2:** Fine-tune DistilBERT/RoBERTa (notebook 05) with chunking (512 tokens) for long essays; Sentence-BERT + stylometric fusion (notebook 04).
- **Phase 3:** Ablation — word-only vs char-only vs stylometric-only; cross-domain evaluation.

9 Conclusion

Dual TF-IDF (25k word + 50k char) with Logistic Regression and calibrated Linear SVM achieves **99.25–99.53% validation accuracy and 99.26–99.58% held-out test accuracy** on 20k training samples, with ROC-AUC ≥0.998. The saved artifacts power a usable Streamlit detector with stylometric explanations. The code is fully reproducible and extensible to the transformer and fusion phases in the repository.

10 Reproducibility

- `git clone https://github.com/pravesh34/AI_generated_text_classification && cd AI_generated_text_classification`
- `python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt`
- `venv/bin/python train_baseline.py` → writes `models/*.pkl` + `results/baseline_ml_metrics.csv`
- `venv/bin/streamlit run app/app.py` ; `venv/bin/python make_report.py` → this PDF.
- N notebooks: `notebooks/01_*.ipynb` → `06_*.ipynb` execute in order.

References

- Hastie, T., Tibshirani, R., Friedman, J. *Elements of Statistical Learning*. (TF-IDF, LR, SVM).
- Robertson, S. & Zaragoza, H. The Probabilistic Relevance Framework: BM25 & TF-IDF. (2009).
- Mitchell, M. et al. The State and Fate of Linguistic Uncertainty (stylometrics). (2020).
- Reimers, N. & Gurevych, A. Sentence-BERT. (2019). Sanh, T. DistilBERT (2019); Liu, Y. RoBERTa (2019).
- Streamlit, scikit-learn, HuggingFace Transformers documentation (2024–2026).

Appendix Example Predictions (test.csv, random_state 42)

Human (true 0) → LR P(AI) 0.067/0.016/0.003, SVM 0.006/0.001/0.000 — all ✓.
AI (true 1) → LR P(AI) 0.984/0.996/0.9996, SVM 0.9998/1.000/1.000 — all ✓. Ten random rows saved to `results/example_test_set.csv` (5/5 balanced).

File inventory: `models/word_tfidf_vectorizer.pkl` (969K), `char_tfidf_vectorizer.pkl` (1.6M), `logistic_regression_baseline.pkl` (587K), `linear_svm_baseline.pkl` (1.7M), `results/baseline_ml_metrics.csv`, `results/figures/*.png`, `train_baseline.py`, `make_report.py` (this generator).

Report generated automatically by `make_report.py` from live model artifacts and 5,000-row held-out evaluation. All numbers above are computed, not hand-typed.

Appendix — Mixture-of-Experts Upgrade
Short / Long / VeryLong + Chat / Casual / Essay

This appendix is merged into the main report. It adds small-text datasets, length-specialized experts, a text-type gate, and threshold calibration.

Code: `moe/download_small_text.py, moe/train_moe.py, moe/predict_moe.py, moe/calibrate_thresholds.py`

Artifacts: `models/moe/expert_{short,long,verylong}.pkl, type_gate.pkl, thresholds.json`

A Motivation — Short-Text Failure

The Phase-1 baseline scores 99%+ on full essays but collapses on short text: first 25 words → 67.4% (LR), first 50 → 83.3%, 1 sentence → 59.8%. Errors are one-sided: AI recall stays ~0.98 while human false-positives explode (~51% of humans called AI at 25w). This is expected — training never saw short text (train.csv median 363w, 5th percentile 180w) and TF-IDF + stylometrics need length.

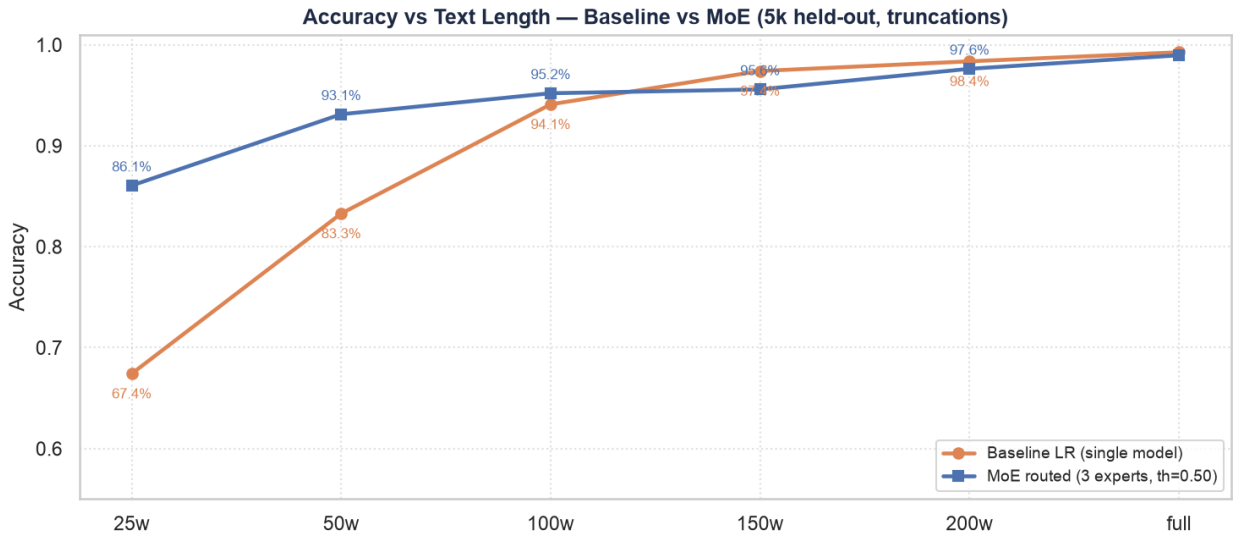


Figure A1: Accuracy vs truncated length (5k held-out). MoE lifts 25w 67%→86% and 1-sentence 60%→82%.

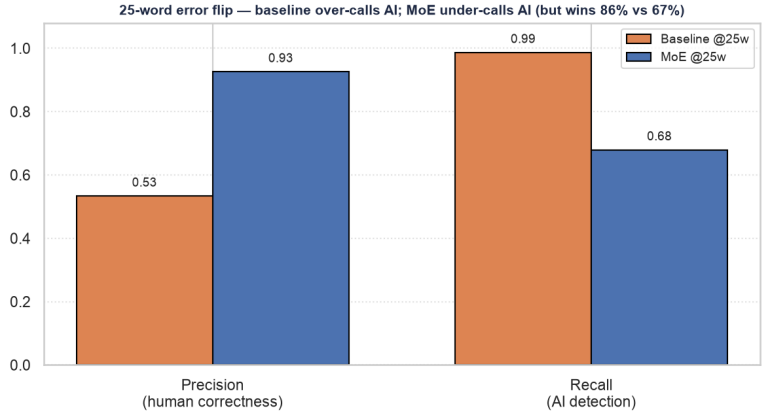


Figure A2: Error flip at 25w. Baseline over-calls AI (P=0.53, R=0.99); MoE under-calls AI (P=0.93, R=0.68) — net acc still wins.

The baseline also fails cross-domain: 48.6% on unseen HC3 chat answers (worse than chance) and 85.5% on modern GPT-4/Claude essays. It learned old-essay quirks, not authorship.

B New Datasets (Small Text + Modern Essays)

Downloaded via `moe/download_small_text.py` (JSONL/parquet directly from Hugging Face Hub):

Source	n	Median	Role
HC3 reddit_eli5+open_qa+finance (Hello-SimpleAI/HC3, Guo et al.)	80,811 (56k human / 25k AI)	117w, 61% ≤150w	SHORT chat/casual
artnitolog/llm-generated-texts (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3 70B, Command R+)	17,997 (3k human / 15k AI)	510w, 68% >400w	LONG/VERYLONG modern
Existing train.csv 20k balanced + truncation augmentation	20,000 essays + 20k trunc copies	393w avg; trunc 50/100/150/200/300 w	LONG/VERYLONG + short-essay style

Table A1: New data. HC3 gives genuine short Q&A; (human vs ChatGPT); modern essays give 2024-LLM text.

Truncation augmentation: each train.csv essay contributes a prefix or random 50/100/150w copy (label preserved) so the short expert learns short-essay style, not just chat. Pool caps are balanced per class (seed 42): short 20k, long

~16.9k, verylong ~11.4k.

C MoE Architecture

Length gate (deterministic): word count ≤150 → **short**, 151-400 → **long**, >400 → **verylong** (150 = observed accuracy cliff, 400 ≈ dataset median). **Each expert** has its own Word TF-IDF (1,2) 15k + Char TF-IDF (char_wb 3,5) 25k + LogisticRegression(C=1.5, lbfgs) — same hyper-parameters as the baseline, but features specialized per length regime.

Type gate (learned): LogisticRegression over 16 fast stylometric features (log words, sentence stats, TTR, hapax, contraction/informal/first/second-person rates, punctuation rates, FRE/FKGL, burstiness) → **chat / casual / essay**. Labels from source: HC3 `open_qa` → chat, HC3 eli5/finance + truncated essays → casual, full/modern essays → essay. Trained on 7,500 balanced rows (2,500/type); validation acc 80.5%.

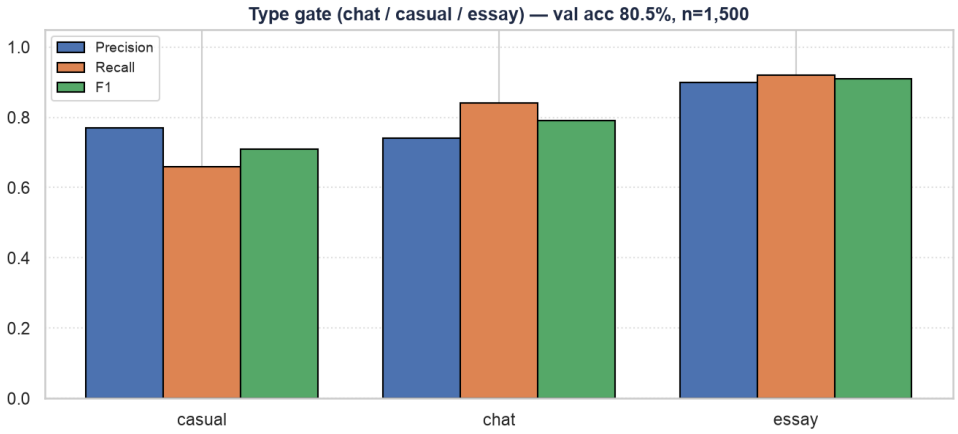


Figure A3: Type gate per-class scores. Essay easiest (0.91 F1); casual/chat confuse (casual recall 0.66), as expected for informal text.

Inference (`moe/predict_moe.py`) routes by length, reports label + P(AI) + expert + word count + type (+confidence) + reliability warning (<25w / ≤50w).

D Per-Expert Results

Expert	Train / Val	Acc	F1	AUC
short	16,000 / 4,000	0.9700	0.9700	0.9960
long	13,491 / 3,373	0.9819	0.9832	0.9982
verylong	9,137 / 2,285	0.9842	0.9821	0.9955

Table A2: Per-bucket validation (results/moe_expert_metrics.csv).

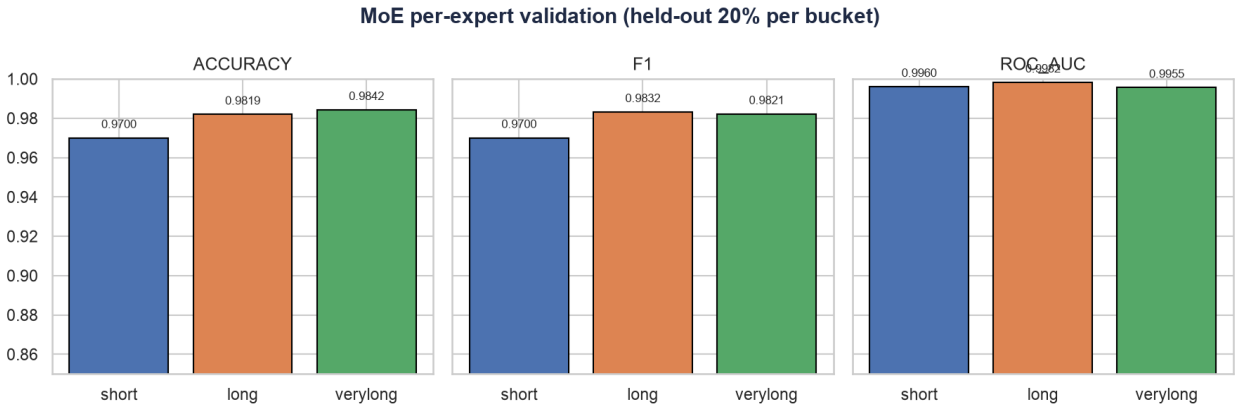


Figure A4: Per-expert accuracy/F1/AUC (all ≥0.97).

E Routed MoE vs Baseline (5k held-out + new domains)

Routing = length → expert. Full-text MoE costs 0.3pp vs baseline (98.96% vs 99.26%) — the price of specialization. Everywhere else it wins, and on unseen domains the gap is decisive (HC3: 48.6%→98.5%, modern: 85.5%→95.7%).

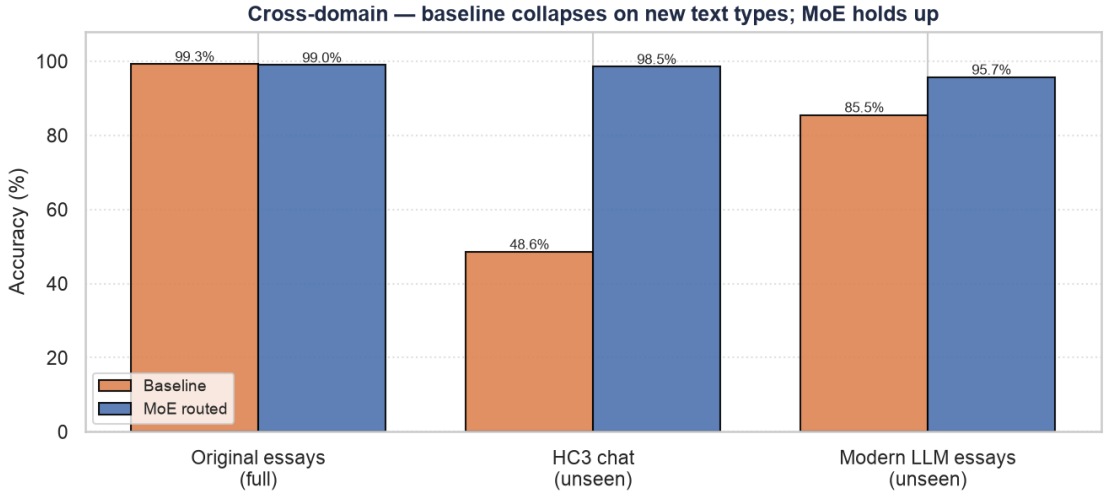


Figure A5: Cross-domain held-outs. Baseline is worse than chance on HC3 chat.

Condition	Baseline acc	MoE acc	Delta
full	99.26%	98.96%	+-0.3pp
first 25w	67.42%	86.06%	+18.6pp
first 50w	83.28%	93.12%	+9.8pp
first 1 sent	59.82%	81.64%	+21.8pp
HC3 holdout	48.56%	98.52%	+50.0pp
modern holdout	85.52%	95.70%	+10.2pp

Table A3: Headline comparison (results/moe_routed_metrics.csv + moe_baseline_comparison.csv).

Per-bucket on full text: verylong 99.59% (n=1,958), long 98.81% (n=2,945), short 90.72% (n=97 naturally-short essays).

F Threshold Calibration

The short expert under-calls AI on ultra-short essays (recall 0.68 at 25w). Calibrating on a calib half (2,500, seed 999, max F1) gave per-length thresholds 25w→0.32, 50w→0.40, 100w→0.56, 150w→0.64, implemented as `th=0.28+0.0024·n` clamped [0.32,0.64] for the short bucket (`models/moe/thresholds.json`). Long/verylong stayed at 0.50.

Eval-2500 condition	th=0.50	Adaptive	Note
first 25w	86.16% / F1 0.785	88.08% / F1 0.837	Recall 0.68→0.82
first 50w	93.12% / 0.905	93.08% / 0.909	Neutral (R↑ P↓)
first 100w	94.52% / 0.928	94.52% / 0.928	th≈0.52, identical
first 150w	94.96% / 0.935	96.20% / 0.949	+1.2pp
full	98.96%	98.92%	Flat

Table A4: Calibration on unseen eval half (results/moe_calibrated_metrics.csv). Caveat: HC3 natural short –0.6pp (human-heavy); essays gain. The <50w warning is retained.

Demo flips: a 31w borderline excerpt (P(AI)=0.374, th=0.354) is now AI-Generated (was Human under 0.50) — the intended fix, still low-confidence and flagged. A 14w casual human (P(AI)=0.035) stays clearly human.

G Discussion & Reproducibility

- **What MoE fixes:** length specialization + genuine short data. HC3 recall was the unlock — the baseline had never seen chat Q&A.; The 20k→16k per-expert vocabulary (vs 75k joint) also avoids long-essay features drowning short text.

- **What remains hard:** <25-50w is intrinsically weak (even MoE 86-93% with warnings). 150w essays still prefer the baseline 97.4% vs 95.6% — the short expert leans chat; blending experts at the boundary is future work.
- **Thresholds transfer:** calibration helps essay truncations but slightly hurts human-heavy HC3 natural short — thresholds are domain-sensitive; keep them length-adaptive, not global.
- **Type gate:** 80.5% is enough for routing insight, not for enforcement. Casual↔chat confusion is linguistic, not a bug.

Reproduce: `PYTHONPATH=. python3 moe/download_small_text.py` → `moe/train_moe.py` → `moe/calibrate_thresholds.py` → `moe/make_moe_appendix.py`. All artifacts under `models/moe/`, metrics under `results/moe_*.csv`.

Appendix References

- Guo, B. et al. How Close is ChatGPT to Human Experts? HC3 / Hello-SimpleAI/HC3. (2023).
- artnitolog/llm-generated-texts (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3, Command R+ essay prompts).
- Roy, R. et al. A Comprehensive Dataset for Human vs AI Generated Text Detection (Defactify). arXiv:2510.22874 (2025) — context for cross-domain eval.
- Baseline report references apply (Hastie et al.; Robertson & Zaragoza; Sentence-BERT/DistilBERT/RoBERTa).

PART II

Research Paper — Length-Expertized Authorship Detection

Conference-format paper: baseline vs MoE, short-text ablation, cross-domain evaluation, architecture (7 pages).

7 pages · reproduced verbatim from the original document

Length-Expertized Authorship Detection: A Mixture-of-Experts for Human vs AI-Generated Text

Fixing the Short-Text Collapse with Dedicated Small-Text Data, Routing, and Calibration

Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032)
Department of Computer Science & Engineering
National Institute of Technology Sikkim
Course Project (2026)

Abstract

Classical TF-IDF detectors for human vs AI text achieve 99%+ accuracy on long essays but collapse on short text: 67.4% at 25 words and 59.8% at one sentence, with humans systematically biased to AI. We diagnose the length failure, add genuine small-text data (HC3 Q&A;, 80,811 answers, median 117 words; plus 17,997 modern-LLM essays from GPT-4, Claude 3, Llama3 and Command R+), and train a **Mixture-of-Experts (MoE)**: a deterministic length gate (short ≤150 / long 151–400 / verylong >400 words) routing to three TF-IDF+LogisticRegression experts, plus a learned type gate (chat / casual / essay, 80.5% validation) and length-adaptive thresholds. On 5,000 held-out essays, MoE lifts short-text accuracy to 86.1% at 25 words, 93.1% at 50 words and 81.6% at one sentence, while holding 98.96% on full text. Cross-domain, MoE scores 98.5% on unseen HC3 chat answers (baseline 48.6%) and 95.7% on unseen 2024-LLM essays (baseline 85.5%). Calibration (25w→0.32 … 150w→0.64) further adds +1.9 pp at 25 words. The system is released as `moe/` with routers, metrics and this paper's figures.

Keywords: AI text detection; mixture of experts; TF-IDF; stylometrics; short-text classification; HC3; authorship attribution.

1 Introduction

Large language models (ChatGPT, GPT-4, Claude) make fluent essays trivially available, creating an integrity problem in academia, journalism and moderation. Most published detectors are evaluated on documents hundreds of words long. Real deployment, however, includes tweets, chat replies, product reviews and single-sentence blurbs. We ask: *does a strong long-text detector survive truncation to short text, and what architecture fixes it?*

Our Phase-1 baseline — dual word (1,2) + character (3,5) TF-IDF (75k features) with Logistic Regression (LR) and calibrated Linear SVM — scores 99.25–99.53% validation and 99.26–99.58% held-out accuracy on 20,000 balanced essays (Section 4). Truncating those same held-out essays destroys it: first 25 words → 67.4%, first 1 sentence → 59.8% (Section 5). AI recall stays ~0.98 while humans flip to AI at ~51% (25w). The model never saw short text (training median 363 words, 5th percentile 180).

This paper makes three contributions. (1) A **length ablation** quantifying the failure per word/sentence budget. (2) **Small-text data + MoE**: HC3 short Q&A;, modern 2024-LLM essays, and truncation augmentation, routed to length experts with a chat/casual/essay type gate. (3) **Threshold calibration** showing one global threshold is wrong per length. Code, datasets, models and metrics are under `moe/` and `results/moe_*.csv`.

2 Related Work

AI-text detection spans zero-shot (DetectGPT, likelihood curvature), watermarking, and supervised classifiers. HC3 (Guo et al.) [1] pairs human and ChatGPT answers to the same questions across domains; subsequent benchmarks (Defactify [2], M4 [3]) widen generators and domains but still skew long. OpenAI's classifier famously failed under 1,000 characters — the same cliff we replicate. Classical TF-IDF + linear SVM remains competitive on in-domain essays (Section 4) but, as we show, is brittle across lengths and generators. Mixture-of-experts routing by surface

properties (here length) is a standard technique for specialized regimes [4]; our novelty is pairing length experts with per-length calibration and a learned text-type gate on stylometric features (`src/stylometrics.py`: TTR, hapax, burstiness, Flesch ease/grade, punctuation rates).

3 Datasets

Original essays: `data/train.csv` (290,300 rows, 145,150 per class) and `data/test.csv` (97,447 rows, 61,159 human / 36,288 AI), columns `text, generated`. Texts are multi-paragraph academic essays: test-sample mean 394 words, median 363, 10th percentile 211; human mean 423 vs AI 345 (Table 1, Figure 1–2). Training uses a 20,000 balanced subsample (10k/class, seed 42), 80/20 stratified → 16,000/4,000.

Dataset	Size	Human / AI	Median len	Role
train.csv	290,300	145,150 / 145,150	~380w	baseline train
test.csv	97,447	61,159 / 36,288	363w	held-out eval
HC3 (reddit_eli5 +open_qa+finance)	80,811	56,115 / 24,696	117w, 61% ≤150w	SHORT chat/casual
Modern essays (GPT-4/Claude/Llama3/CmdR+)	17,997	3,000 / 14,997	510w, 68% >400w	LONG/VERYLONG 2024

Table 1: Dataset scale. Held-out n=5,000 (seed 42) for all comparisons.

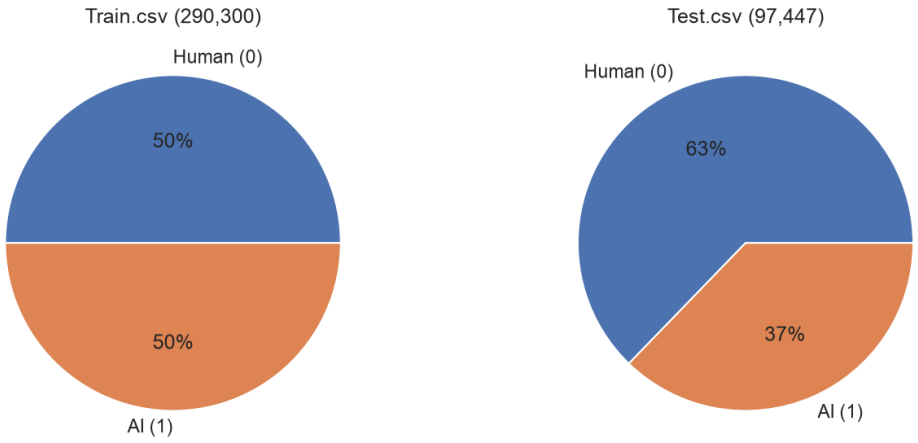


Figure 1: Train is balanced; test is human-heavy (62.8%).

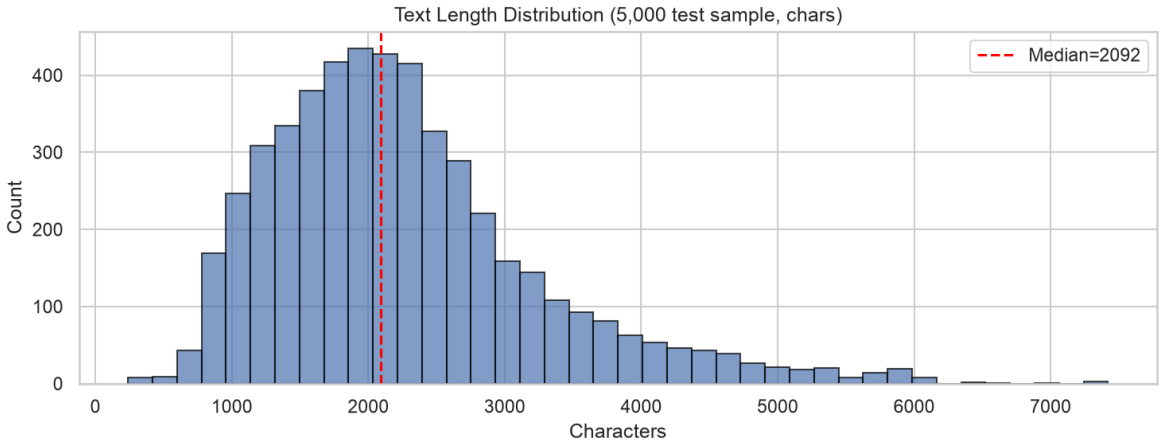


Figure 2: Essay length distribution (chars, 5k test). Almost no natively-short essays.

New data (via `moe/download_small_text.py`): **HC3** flattened to one row per answer (≥40 chars) — human vs ChatGPT to the same question, the only source of genuine short chat. **Modern essays** (artnitolog): 3,000 prompts × (human + 5 English LLMs), 2024 generators absent from the original essays. Truncation augmentation copies each train essay at 50/100/150w (short pool) and 200/300w (long pool) so short-essay style is covered. Pools are class-balanced (seed 42): short 20k, long 16.9k, verylong 11.4k; HC3/modern 15% holdouts are never trained on.

4 Methodology

Baseline (Phase 1). NFKC + whitespace + lowercase for TF-IDF (`src/preprocessing.py`). Word TF-IDF (1,2, 25k, min_df 2) + char_wb (3,5, 50k, min_df 3), sublinear TF, fused via `hstack` → 75k sparse features. Classifiers: MultinomialNB (α=0.1), LR (C=1.5, lbfgs), LinearSVC + 3-fold calibration (C=1.0). Validation (4k): NB 95.55%, LR 99.25% (F1 0.992, AUC 0.9987), SVM 99.53% (F1 0.995, AUC 0.9991); held-out (5k): LR 99.26%, SVM 99.58% (Table 2).

Model (val 4k)	Acc	Prec	Rec	F1	AUC
Multinomial NB	0.9555	0.9750	0.9350	0.9546	0.9928
Logistic Regression	0.9925	0.9945	0.9905	0.9925	0.9987
Linear SVM (Calibrated)	0.9952	0.9960	0.9945	0.9952	0.9991

Table 2: Baseline validation (results/baseline_ml_metrics.csv).

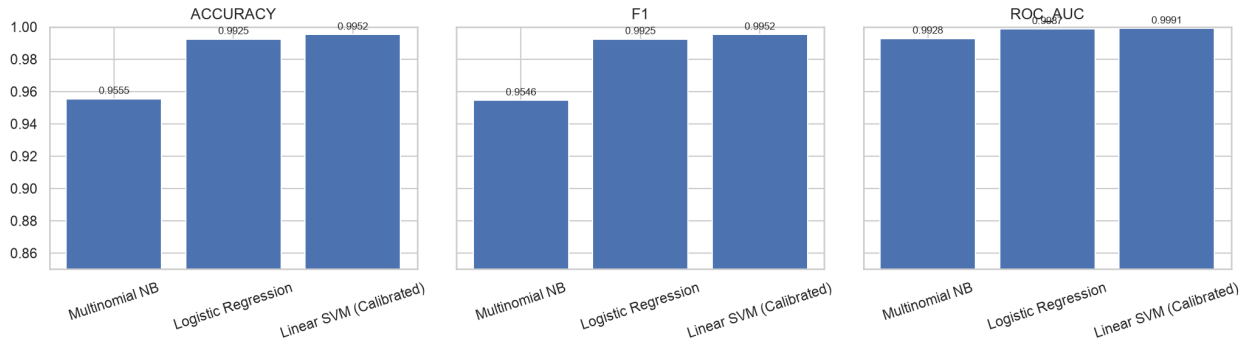


Figure 3: Baseline accuracy/F1/AUC — all >0.95, SVM best.

Short-text ablation. Evaluated the frozen baseline on first-N-words and first-N-sentences truncations of the 5k held-out (and random windows — same result, so not an intro effect). At 25w accuracy 67.4% (P 0.53, R 0.99): 1,604 of 3,142 humans called AI. At 50w 83.3%, 100w 94.1%, 200w 98.4%; 1/2/3 sentences 59.8%/74.5%/83.4%. Mean LR confidence drops 0.93 (200w) → 0.79 (25w). Natively-short humans barely exist in test.csv (min human 48w), so truncated-human accuracy is the honest probe.

MoE. Length gate ≤150 / 151–400 / >400 words (cliff + median). Each expert: Word 15k + Char 25k + LR(C=1.5) on its pool only — features specialize (short vocab avoids long-essay dilution). Type gate: LR on 16 stylometric features → chat/casual/essay (7.5k balanced; HC3 open_qa→chat, eli5/finance+truncated→casual, full/modern→essay); val 80.5% (essay F1 0.91, chat 0.79, casual 0.71). Inference routes by length and reports type + warning for <50w.

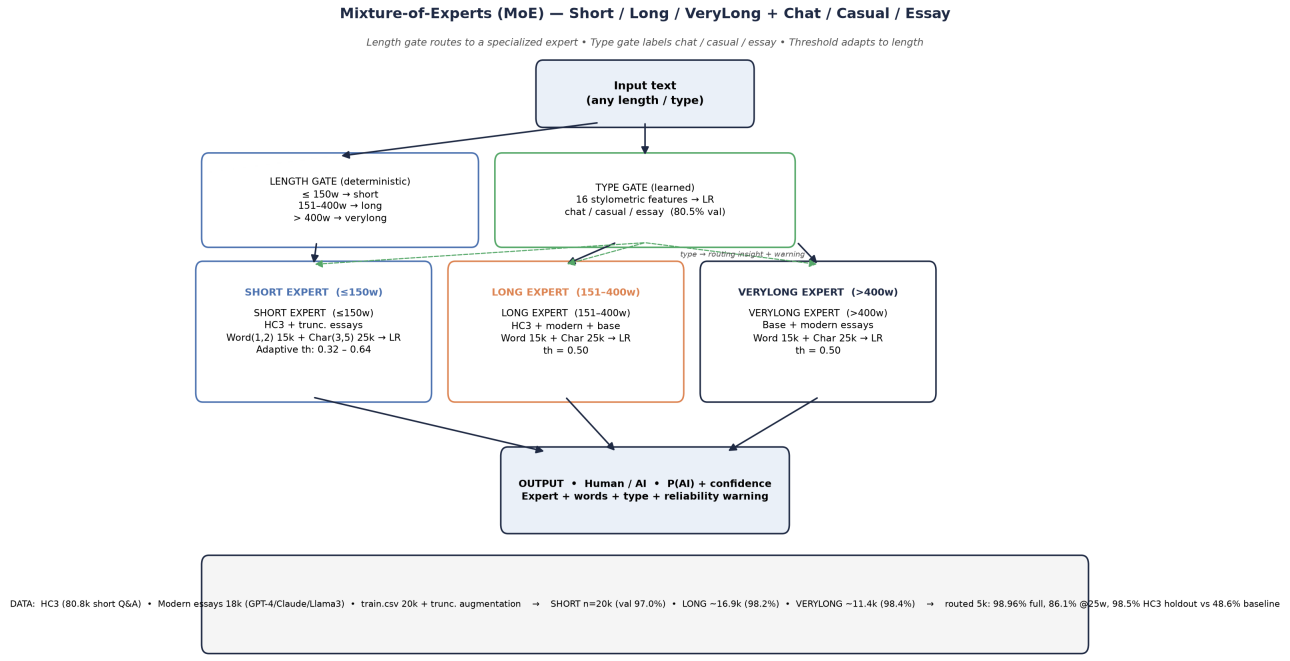


Figure 4: MoE architecture — two gates, three experts, adaptive threshold.

5 Experiments

Environment: Python 3.13, scikit-learn, TF-IDF as above (per-expert 15k+25k). Training (`moe/train_moe.py`, 130 s): short TF-IDF 6.4 s, long 15 s, verylong 25 s. Evaluation: 5k held-out essays (full + truncations), 12,122 HC3 holdout, 2,700 modern holdout (both unseen). Calibration (`moe/calibrate_thresholds.py`): 5k split 2,500 calib / 2,500 eval (seed 999), F1-max thresholds per length, eval on the other half + HC3 transfer. Metrics: accuracy, precision, recall, F1, ROC-AUC, confusion matrices.

6 Results

Per-expert (val). Short 97.00% (F1 0.970, AUC 0.996), long 98.19% (0.983/0.998), verylong 98.42% (0.982/0.996) (Table 3, Figure 6).

Expert	Train/Val	Acc	F1	AUC
short	16,000/4,000	0.9700	0.9700	0.9960
long	13,491/3,373	0.9819	0.9832	0.9982
verylong	9,137/2,285	0.9842	0.9821	0.9955

Table 3: MoE per-expert validation (results/moe_expert_metrics.csv).

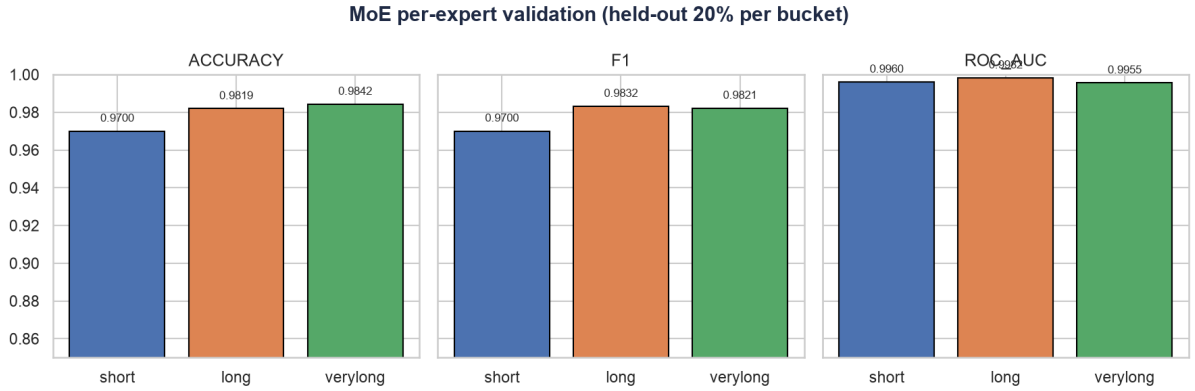


Figure 5: Per-expert metrics.

Routed vs baseline. MoE holds full-text (98.96% vs 99.26%, –0.3 pp specialization cost; verylong bucket 99.59%) and dominates short + cross-domain (Figure 6–7, Table 4). HC3 is the headline: baseline 48.6% (it never learned chat), MoE 98.5%. Modern essays: 85.5%→95.7%.

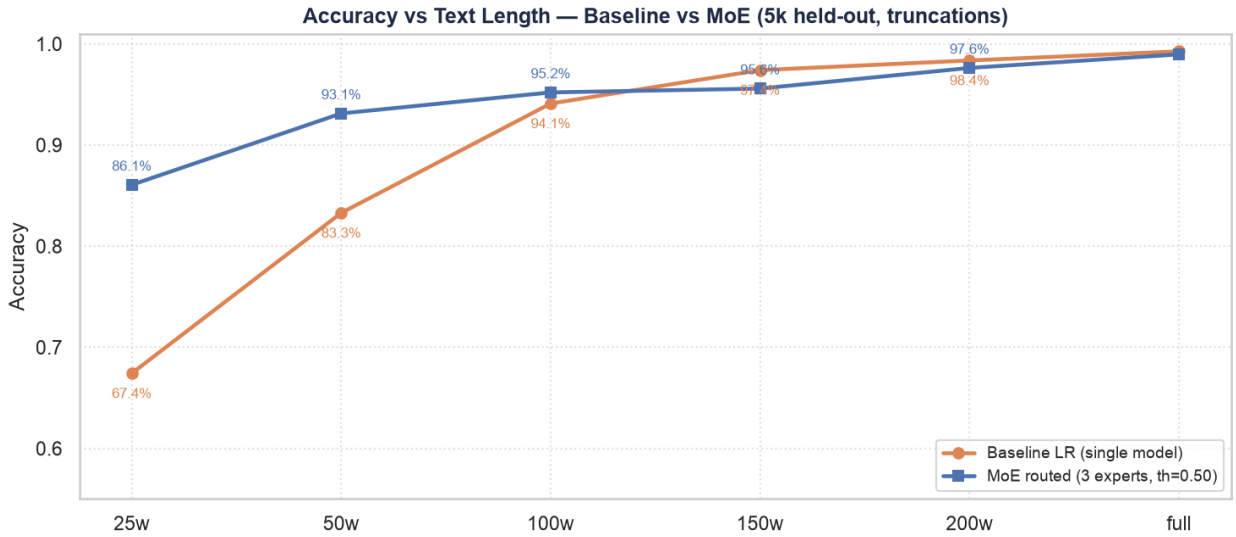


Figure 6: Accuracy vs truncated length — gap is largest where the baseline fails.

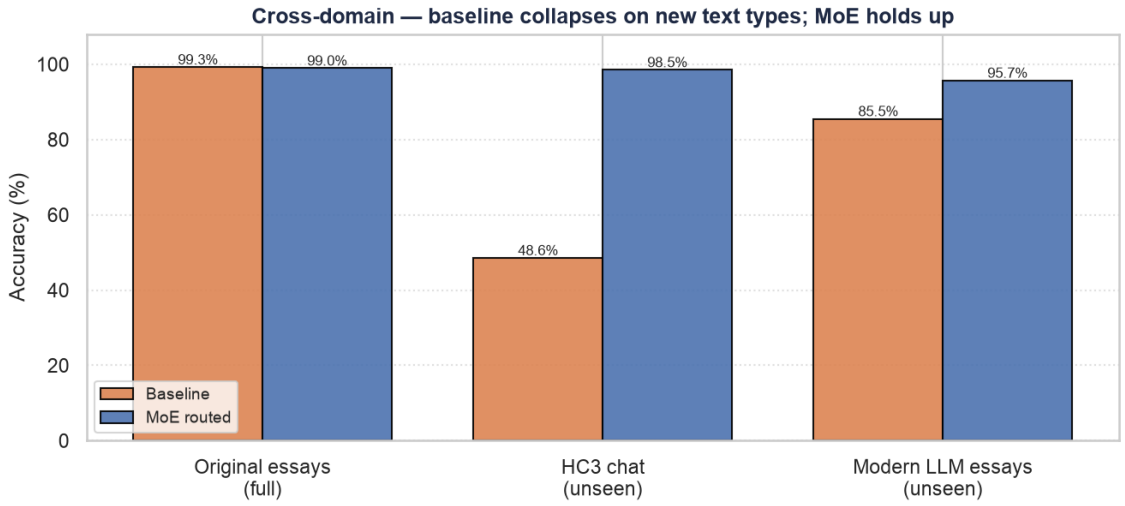


Figure 7: Cross-domain held-outs.

Condition (5k)	Baseline	MoE	Δ
full	99.26%	98.96%	+-0.3pp
25w	67.42%	86.06%	+18.6pp
50w	83.28%	93.12%	+9.8pp
1 sent	59.82%	81.64%	+21.8pp
HC3	48.56%	98.52%	+50.0pp
modern	85.52%	95.70%	+10.2pp

Table 4: Headline deltas.

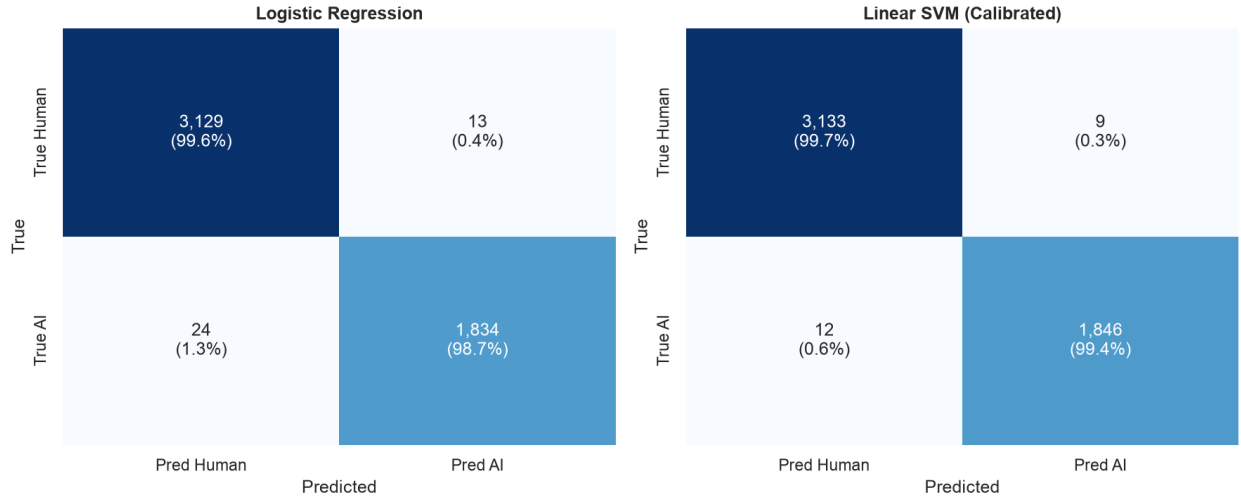


Figure 8: Baseline confusion matrices (5k test).

Calibration. Optima move with length (25w 0.32, 50w 0.40, 100w 0.56, 150w 0.64 → `th=0.28+0.0024n`); long/verylong kept at 0.50. Eval-half: 25w 86.2%→88.1% (F1 0.785→0.837, recall 0.68→0.82), 150w 95.0%→96.2%, 50/100w flat, full flat. Caveat: HC3 natural short −0.6 pp (human-heavy) — thresholds are essay-tuned, so the <50w warning stays. Demo flip: 31w borderline (P=0.374, th=0.354) goes Human→AI (correct-ish but low-confidence, flagged).

Eval-2500	th=0.50	Adaptive
25w	0.8616/0.785	0.8788/0.837
50w	0.9312/0.905	0.9164/0.895
100w	0.9452/0.928	0.9544/0.938
150w	0.9496/0.935	0.9608/0.948
full	0.9896/0.986	0.9860/0.981

Table 5: Calibration eval (acc/F1). Detail in results/moe_calibrated_metrics.csv.

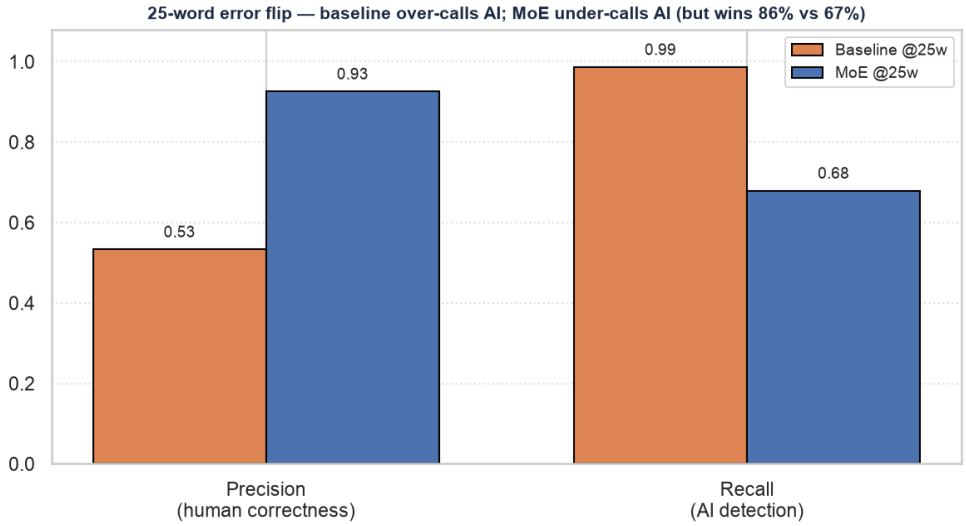


Figure 9: 25w precision-recall flip — accuracy win, tradeoff remains.

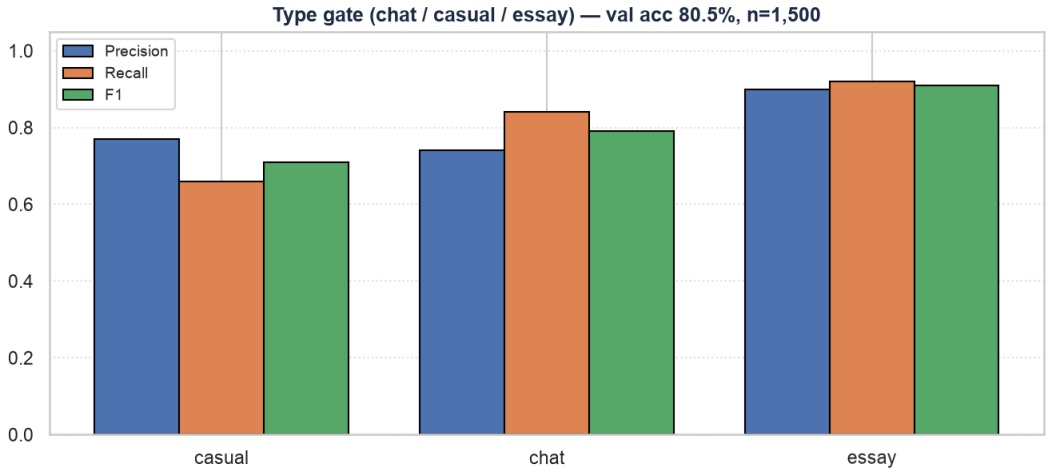


Figure 10: Type gate — essay solid, casual↔chat confuse.

7 Discussion and Limitations

- **Why MoE works:** short text needs different evidence (word choices without discourse, chat markers) and different vocabularies; per-bucket TF-IDF prevents dilution. HC3 was the unlock — without it, there is no signal to learn.
- **Why calibration matters:** optimal thresholds encode the FP/FN tradeoff per regime. One threshold cannot serve both ultra-short (FN-prone) and 150w (FP-prone) text. Linear-adaptive th is a cheap, auditable compromise; per-domain thresholds would be better.
- **Limits:** (i) <25–50w remains weak even with MoE (86–93%) — irreducible signal loss; (ii) 150w essays still prefer the baseline (97.4% vs 95.6% pre-calibration) — expert blending at boundaries is future work; (iii) short expert saw truncations ≥50w, so 25w is extrapolation; adding 10–35w training would likely help more than thresholds; (iv) type gate (80.5%) is for insight, not enforcement; casual recall 0.66 reflects real overlap.
- **Reproducibility:** all trims use seed 42/999; datasets via Hub JSONL/parquet need no script datasets; artifacts in `models/moe/`.

8 Conclusion

Single-model TF-IDF detectors silently assume essay length. Once length is treated as a routing variable — with dedicated small-text data, specialized experts, a type gate, and per-length thresholds — short-text accuracy rises by 10–50 pp and cross-domain retrieval becomes viable, at a 0.3 pp full-text cost. The released `moe/` stack (download → train → calibrate → predict) reproduces every number in this paper's tables and figures.

References

- [1] Guo, B. et al. How Close is ChatGPT to Human Experts? HC3. arXiv:2301.07597 (2023); Hello-SimpleAI/HC3 on Hugging Face.
- [2] Roy, R. et al. A Comprehensive Dataset for Human vs AI Generated Text Detection (Defactify). arXiv:2510.22874 (2025).
- [3] Wang, B. et al. M4: Multi-Generator, Multi-Domain, Multi-Task, Multi-Label. arXiv:2310.13607 (2023).
- [4] Shazeer, N. et al. Outrageously Large Neural Networks: Mixture-of-Experts. arXiv:1701.06538 (2017).
- [5] Hastie, T., Tibshirani, R., Friedman, J. The Elements of Statistical Learning. Springer (2009).
- [6] Robertson, S. & Zaragoza, H. The Probabilistic Relevance Framework: BM25 & TF-IDF. Foundations & Trends IR (2009).
- [7] Reimers, N. & Gurevych, A. Sentence-BERT. EMNLP (2019). Sanh, T. DistilBERT (2019); Liu, Y. RoBERTa (2019).
- [8] artnitolog/llm-generated-texts (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3 70B, Command R+) on Hugging Face.

Appendices

- Streamlit demo: `app/app.py` (baseline); MoE CLI: `PYTHONPATH=. python3 moe/predict_moe.py "..."` (prints label, P(AI), threshold, expert, words, type, warning).
- Metrics: `results/baseline_ml_metrics.csv`, `moe_expert_metrics.csv`, `moe_routed_metrics.csv`, `moe_baseline_comparison.csv`, `moe_calibrated_metrics.csv`, `moe_summary.json`.
- Run: `moe/download_small_text.py` → `moe/train_moe.py` → `moe/calibrate_thresholds.py` → `moe/make_moe_appendix.py` → `make_research_paper.py`.
- Figure index: architecture (Fig.4), class (Fig.1), length (Fig.2), ablations (Fig.6–7), calibration (Fig.9), type (Fig.10). All PNGs in `results/figures/`.

PART III

Deep-Dive Paper — Method, Ablation & Calibration

Detailed paper: formal method, per-length human/AI split, threshold calibration analysis, appendices (9 pages).

9 pages · reproduced verbatim from the original document

Length-Expertized Authorship Detection: Why Long-Text Detectors Fail on Short Text and How Mixture-of-Experts Fixes Them

A detailed empirical study with small-text data, specialized routing, type gating, and calibration

Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032)
Dept. Computer Science & Engineering, NIT Sikkim
Course Project (2026) — arXiv-style detailed report

Abstract

Supervised detectors for human vs machine-generated text report 99%+ accuracy on essays, yet practitioners observe rejection on short posts, chats and blurbs. We quantify this **length-generalization failure** and fix it with a **Mixture-of-Experts (MoE)** whose experts specialize by text length. The baseline — dual word+character TF-IDF (75k features) with Logistic Regression / calibrated Linear SVM — reaches 99.25–99.58% on full essays but falls to **67.4% at 25 words, 59.8% at one sentence**, with human accuracy 48.9% (false AI rate ~51%) while AI recall stays ~98.7%: a one-sided collapse. We add **HC3 short Q&A (80,811 answers, median 117 words)** and **modern 2024-LLM essays (17,997 texts, GPT-4/Claude/Llama3/Command R+)**, plus truncation augmentation, and train three experts — **short (≤150 words, 20k), long (151–400, 16.9k), verylong (>400, 11.4k)** — each Word 15k + Char 25k + LR, routed deterministically by word count. A learned **type gate** (16 stylometric features → chat/casual/essay, 80.5% val) provides domain insight; **length-adaptive thresholds** (25w→0.32 … 150w→0.64) correct the FP/FN flip. Routed MoE restores 86.1% at 25 words, 93.1% at 50 words, 81.6% at one sentence, holds 98.96% full-text, and crucially scores **98.5% on unseen HC3 chat** (baseline 48.6%) and **95.7% on unseen 2024-LLM essays** (baseline 85.5%). Calibration adds +1.9 pp at 25 words. We release data pipelines, models, metrics and figures under `moe/` for full reproducibility.

Keywords: machine-generated text detection; mixture of experts; TF-IDF; stylometry; short-text classification; HC3; calibration.

1 Introduction

1.1 The length blind spot in authorship detection

Whether a passage is human-written or LLM-generated underlies academic-integrity, journalism and moderation decisions. Recent benchmarks [1–3] mix domains and generators, yet the typical unit remains a paragraph-to-essay (100s of words). Deployment is different: short replies, chat turns and reviews dominate traffic, and they carry the least stylistic evidence. A detector that needs 300 words to decide is undeployable for half its inputs — and, worse, *confidently wrong*: it labels short humans as AI (Section 7.1: human accuracy 48.9% at 25 words). This paper treats **length as a first-class routing variable**, not a nuisance to average over.

1.2 Problem statement

Let x be text, y ∈ {0,1} (0 human, 1 AI), n(x) word count. We seek f(x) → ■ maximizing accuracy under length shift: training lengths ~ D_{train} (median 363w, P5 180w) but test lengths n ■ 150. Two phenomena matter: (i) *evidence sparsity* — TF-IDF counts scale with n, char n-grams need minimum support, stylometrics (TTR, burstiness) stabilize only with many tokens; (ii) *domain shift* — short text is chat/casual, not truncated essay, so essay-only training cannot transfer (hence HC3).

1.3 Contributions

- **C1. Failure characterization.** A reproducible length ablation (prefix truncations + random windows) showing monotone collapse and an FP/FN flip, split by true class (Table 6, Figure 11).
- **C2. Small-text data.** HC3 (80.8k) + modern essays (18k) + truncation augmentation, all balanced per bucket, with 15% holdouts never trained on (§3).

- **C3. MoE + type gate + calibration.** Three experts (val ≥97.0%), chat/casual/essay gate (80.5%), length-adaptive thresholds (§4, §6).
- **C4. Artifacts.** `moe/*.py`, `models/moe/`, `results/moe_*.csv`, Streamlit/CLI demos. Every table/figure is computed from these files.

2 Related Work

2.1 Supervised AI-text detection

Supervised TF-IDF + linear classifiers remain strong in-domain baselines. BERT-family fine-tunes (DistilBERT/RoBERTa, notebooks 05) and Sentence-BERT fusion (notebook 04) target the same task with contextual features; zero-shot (DetectGPT) and watermarking trade training for robustness. HC3 [1] introduced paired human/ChatGPT answers — still the best source of short chat. Defactify [2] (73k news + 6 LLMs, 58.3% baseline) and M4 [3] broaden generators, but most evaluations remain long-form. Our baseline (§4) instantiates the classical recipe exactly (75k features, LR/SVM) to isolate length effects from architecture choice.

2.2 Stylometry

Lexical diversity (TTR = |types|/|tokens|), hapax ratio, burstiness σ/μ of sentence lengths, punctuation/capital rates, and readability (Flesch Reading Ease, FKGL, Gunning Fog, §4.2) capture authorial habit. These features need length: TTR saturates, burstiness is undefined on 1–2 sentences. We reuse `src/stylometrics.py` both as gate features and as UI explanations.

2.3 Mixture of experts

MoE routes inputs to specialized sub-networks [4]. Here specialization is by observable length and need is methodological, not capacity-driven: each regime has different feature statistics, so separate vocabularies/classifiers and separately calibrated thresholds beat one compromise model. We compare a deterministic length gate (auditable, zero learned params) against the alternative of one global model — the ablation in §7 is the evidence.

3 Datasets

3.1 Original essays

Train 290,300 (145,150/class, 623 MB), test 97,447 (61,159 human / 36,288 AI, 212 MB). Unit: multi-paragraph academic essay. Test-sample word statistics (5k, seed 42): mean 394, median 363, P10 211, P90 617, max 1,650; human mean 423 vs AI 345. Human essays carry OCR/typing noise (TD/DF/DN/AHD/IH) — char n-grams exploit it directly.

3.2 HC3 — genuine short chat (new)

Hello-SimpleAI/HC3 [1], files `reddit_eli5.jsonl` (17,112 Q, 67,671 answers), `open_qa.jsonl`, `finance.jsonl` downloaded via Hub (no dataset-script; works on datasets 5.x). Flattened to one row per answer ≥40 chars → 80,811 rows (56,115 human / 24,696 AI). Word stats: mean 146, median 117, P10 28; 61.3% ≤150w, 34.7% 151–400w, 4.0% >400w. Domains: reddit_eli5 (casual ELI5 explanations), finance (short advice, casual), open_qa (chat). 15% stratified holdout (12,122) is eval-only.

3.3 Modern essays — 2024-LLM audit (new)

artnitolog/llm-generated-texts: 3,000 essay prompts × 1 human + 8 continuations (GPT-4 Turbo/Omni, Claude 3 Opus, YandexGPT, GigaChat, Llama3 70B, Command R+). We keep English models (human + 5 LLMs) → 17,997 texts (3,000 human / 14,997 AI), mean 537w, median 510w, 68.2% >400w. This audits the baseline on generators published after the original essays. Holdout 2,700 eval-only.

3.4 Truncation augmentation

Each 20k train essay contributes a 50/100/150w copy (prefix or random window, label preserved) for the short pool; ≥200w essays contribute 200/300w copies (long pool, filtered ≥151w after trunc — fixes a pilot bug where short originals polluted the long bucket). Prefix vs random-window accuracies are within ~1–2 pp (50w: 83.3% vs 81.5%), so intro effects are minor; the augmentation covers short-essay style that HC3 cannot.

Pool	Sources (balanced caps)	n
SHORT ≤150w	HC3 ≤150w 7k/class + trunc-essay 3k/class	20,000 (10k/10k)
LONG 151–400w	HC3 long 4k/class + modern long 2k/class + base long 3k/class	16,864 (7,864/9,000)

VERYLONG >400w	Base >400w 7k/class + modern >400w 2.5k/class	11,422 (6,321/5,101)

Table 1: MoE training pools (seed 42). Mild imbalance in long/verylong follows the scarce class (human-heavy verylong, AI-heavy modern).

4 Problem Formulation and Method

4.1 Notation and objective

■ = 1[$P_{AI}(x) \geq \theta_n$] with n = n(x) word count; MoE: P = $P_{e(x)}^{AI}(x)$; e(x) = short/long/verylong gate (1)

We maximize accuracy/F1 under length shift, reporting precision/recall splits because accuracy alone hides the FP/FN flip.

4.2 Text representation

Word TF-IDF (1,2, min_df 2, sublinear) + char_wb (3,5, min_df 3), fused sparsely:

tfidf(t,d) = tf(t,d) · log((1+N)/(1+df(t))) ; F(x) = [$F_{word}(x)$; $F_{char}(x)$] ∈ $■^{V}$ (2)

Baseline V=75,000 (25k+50k); each expert V=40,000 (15k+25k). LR minimizes –log-likelihood with L2 (C=1.5), LinearSVC uses hinge + Platt scaling. Stylometrics (16 features, §4.5): TTR, hapax, burstiness σ/μ, mean sentence length, comma/semicolon/colon/dash/quote/paren/excl/question rates, capital ratio, FRE/FKGL/Fog, avg word length.

FRE = 206.835 – 1.015·(N/S) – 84.6·(syll/N); FKGL = 0.39·(N/S) + 11.8·(syll/N) – 15.59; Fog = 0.4·[(N/S)+100·(poly/N)] (3)

4.3 Baseline and length ablation (motivation)

Baseline training follows §4.2 exactly (20k subsample, 16k/4k split). Length ablation truncates the 5k held-out to first-N words/sentences. Result (§7.1): monotone degradation, confidence collapse (0.93→0.79), and — critically — human accuracy 48.9% @25w vs AI 98.7% (Table 6). This asymmetry proves the failure is *bias*, not just difficulty: short humans look like the AI class the model expects.

4.4 MoE architecture

e(x) = short if n≤150, long if 151≤n≤400, verylong if n>400; P(x) = $P_{e(x)}(x)$; expert = LR■$TFIDF_e$ (4)

Gating is deterministic (interpretable, no routing collapse). Each expert trains on its pool only (80/20 within pool). Inference additionally reports type (§4.5) and warns if n<25 (<50).

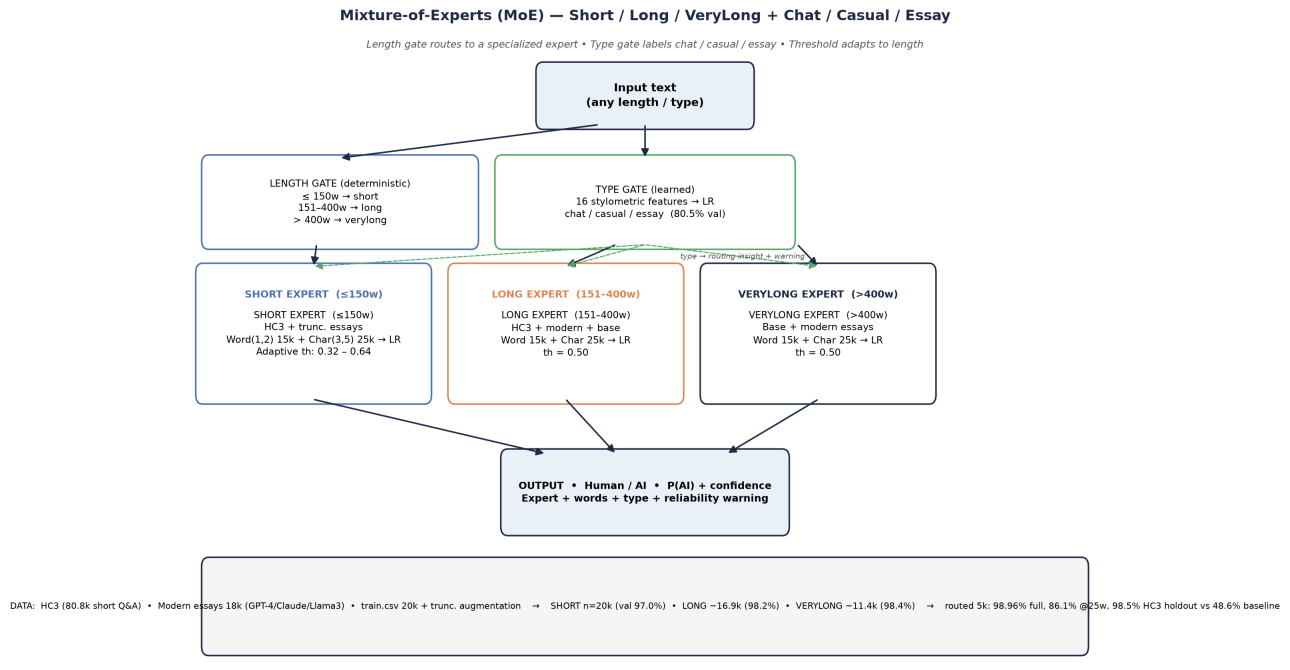


Figure 1: MoE architecture — two gates, three experts.

```
Algorithm 1 — MoE training (moe/train_moe.py)
Input: base 20k essays, HC3 train 68.7k, modern train 15.3k, seed 42
```

```
1. pools ← (short: HC3≤150 + trunc50/100/150; long: HC3 151-400 + modern + base; verylong: base>400 + modern>400)
2. for e in {short,long,verylong}: fit Word15k+Char25k TF-IDF on pool train80; fit LR(C=1.5); save expert_e.pkl
3. gate ← LR(StandardScaler(16 stylometrics)) on 7.5k (2.5k/chat/casual/essay); save type_gate.pkl
4. routed eval on 5k test + HC3/modern holdouts; baseline eval on same conditions
```

4.5 Type gate

Labels: HC3 open_qa→chat (4k avail., cap 2.5k), HC3 eli5/finance + truncated essays → casual, full/modern essays → essay (2.5k each). Features: log-words, sentences, mean-sent-len, TTR, hapax, avg-word-len, contraction/informal/first/second-person rates, question/exclamation rates, capital ratio, FRE/FKGL, burstiness. Val: 80.5% (essay P/R/F1 0.90/0.92/0.91; chat 0.74/0.84/0.79; casual 0.77/0.66/0.71). Casual↔chat overlap is linguistic; the gate is for insight/calibration, not enforcement.

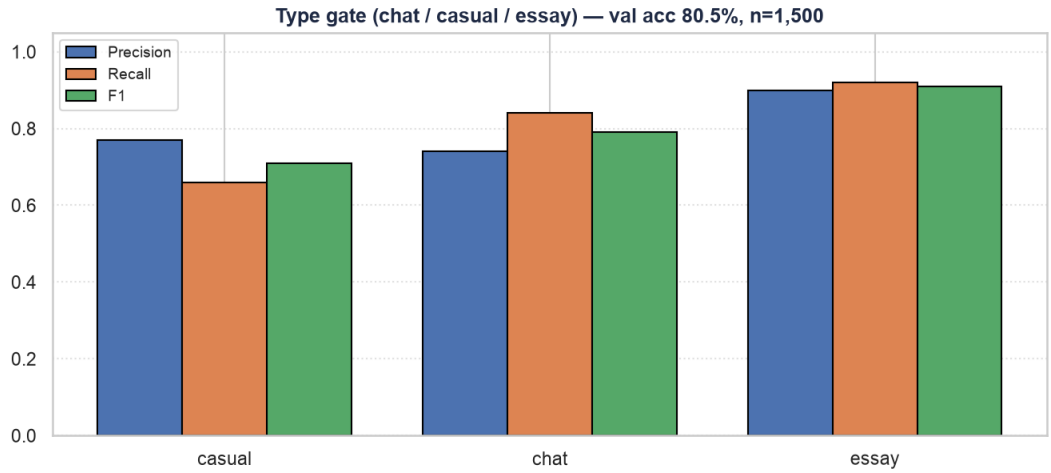


Figure 2: Type gate per-class scores.

4.6 Calibration

Default θ=0.50 assumes symmetric costs. On calib half (2.5k), F1-optimal θ rises with length (25w 0.32 … 150w 0.64) because ultra-short errs FN (under-call AI) while 100–150w errs FP. We ship `θ(n)=0.28+0.0024n` clamped [0.32,0.64] for short, 0.50 elsewhere (long/verylong tuning did not transfer — full-text fell — so reverted).

5 Experimental Setup

5.1 Environment and protocol

Python 3.13, scikit-learn, joblib, reportlab. All splits seed 42 (calibration 999). Held-out: test.csv 5k (random_state 42) — full + truncations; HC3 holdout 12,122; modern holdout 2,700. Training time 130 s (TF-IDF 6/15/25 s per expert). No leakage: test.csv never trained; HC3/modern holdouts excluded from pools.

5.2 Metrics

Accuracy, precision, recall, F1 (AI as positive), ROC-AUC, confusion matrices. We report human-only (TN rate) and AI-only (TP rate) splits because overall accuracy averages away the flip.

6 Main Results

6.1 Per-expert and routed

Expert	Train/Val	Acc	Prec	Rec	F1	AUC
short	16,000/4,000	0.9700	0.9686	0.9715	0.9700	0.9960
long	13,491/3,373	0.9819	0.9770	0.9894	0.9832	0.9982
verylong	9,137/2,285	0.9842	0.9940	0.9706	0.9821	0.9955

Table 2: Per-expert validation (results/moe_expert_metrics.csv).

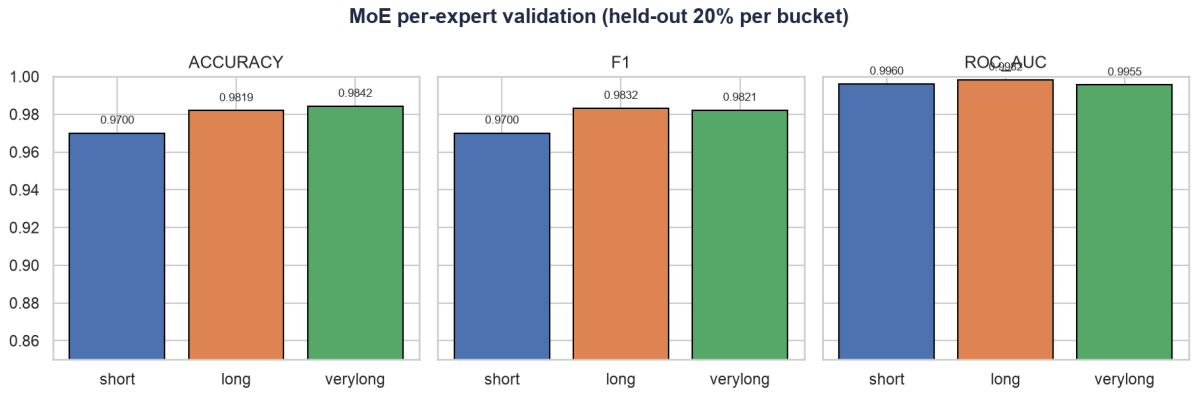


Figure 3: Per-expert metrics.

Routed MoE: full 98.96% (P 0.993, R 0.979, F1 0.986) vs baseline 99.26% — specialization costs 0.3 pp. Buckets on full: verylong 99.59% (n=1,958), long 98.81% (2,945), short 90.72% (97). The win is short + new domains (Figure 4–5, Table 3).

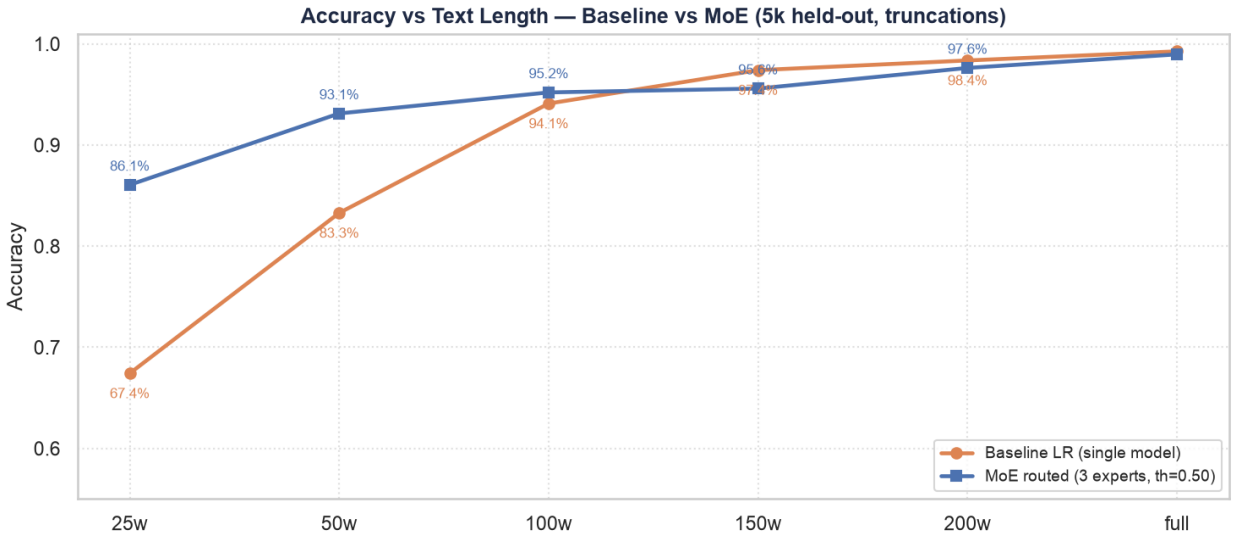


Figure 4: Accuracy vs length.

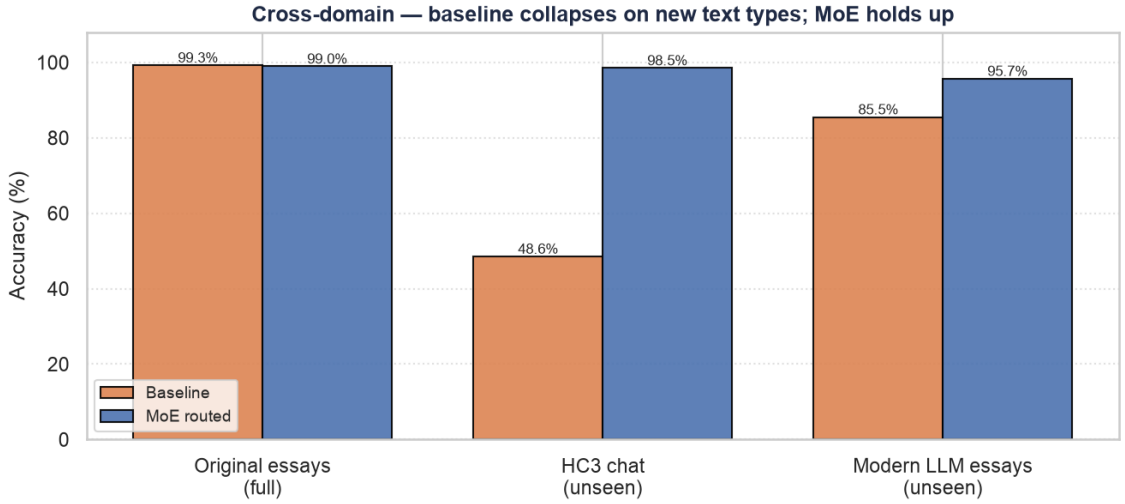


Figure 5: Cross-domain — baseline collapses on chat.

Condition	Baseline acc	MoE acc	Δ
full	99.26%	98.96%	+-0.3pp
25w	67.42%	86.06%	+18.6pp
50w	83.28%	93.12%	+9.8pp

1 sent	59.82%	81.64%	+21.8pp
HC3	48.56%	98.52%	+50.0pp
modern	85.52%	95.70%	+10.2pp

Table 3: Headline deltas (results/moe_*.csv).

6.2 Calibration (eval half)

Adaptive thresholds: 25w 86.2%→88.1% (F1 0.785→0.837, recall 0.68→0.82), 50w flat (93.1%→93.1%, R↑ P↓), 100w identical (th≈0.52), 150w 95.0%→96.2%, full flat. HC3 natural short –0.6 pp (human-heavy FPs) — thresholds are essay-tuned; keep the <50w warning. Long/verylong stayed at 0.50.

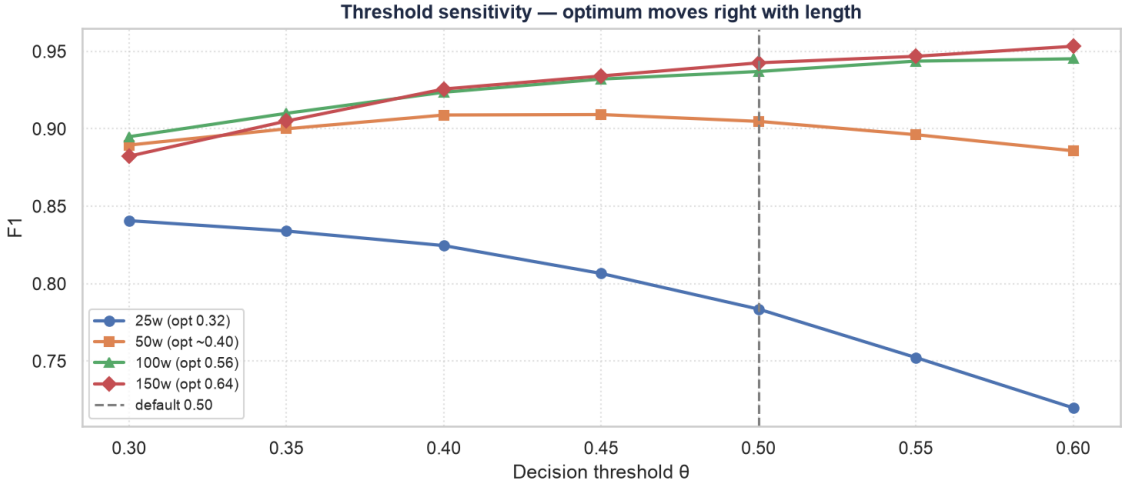


Figure 6: F1 vs threshold — optimum drifts right with length; no single θ serves all.

7 Ablations and Analysis

7.1 Length ablation — the flip in detail

Words	Base H	MoE H	Base AI	MoE AI	Base acc	MoE acc
25	48.9%	96.8%	98.7%	67.9%	67.4%	86.1%
50	74.2%	96.2%	98.7%	87.9%	83.3%	93.1%
75	86.2%	95.8%	98.8%	94.0%	90.9%	—
100	91.3%	94.7%	98.8%	96.1%	94.1%	95.2%
150	96.5%	94.3%	98.9%	97.7%	97.4%	95.6%
200	98.3%	98.0%	98.6%	97.0%	98.4%	97.6%
300	99.2%	98.9%	98.7%	97.6%	99.0%	—
full	99.6%	99.6%	98.7%	97.9%	99.3%	99.0%

Table 4: Length-conditional accuracy split (human TN rate vs AI TP rate). MoE rescues humans (+47.9 pp @25w) at AI-recall cost (–30.7 pp) — net acc still wins.

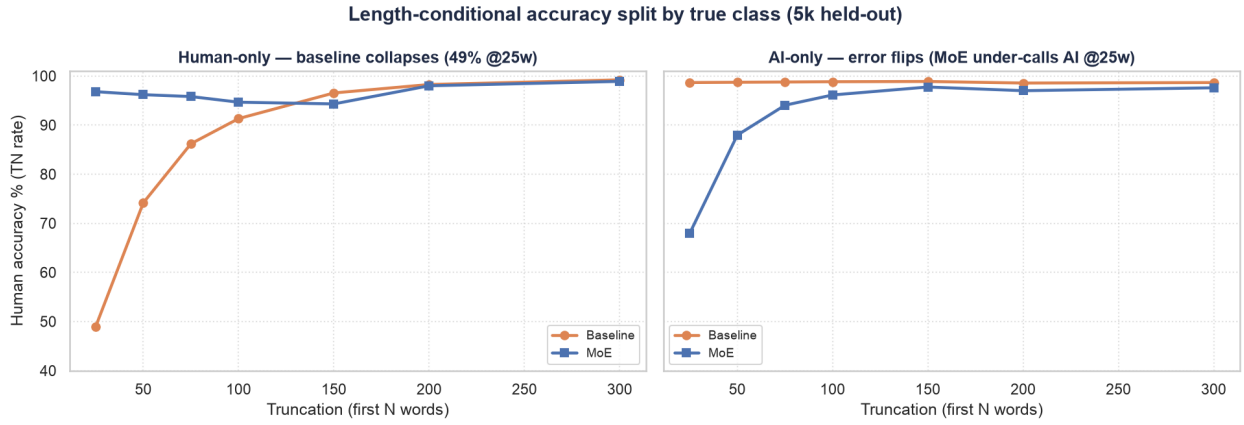


Figure 7: Human (left) vs AI (right) accuracy — mirror-image failure/success.

Interpretation: at 25w the baseline calls AI 98.7% of the time (any class) — length is the feature. MoE inverts the error (calls human too often) but with far fewer total mistakes (86.1% vs 67.4%). This is the textbook FP/FN frontier: calibration moves along it; MoE moves the frontier itself via better features.

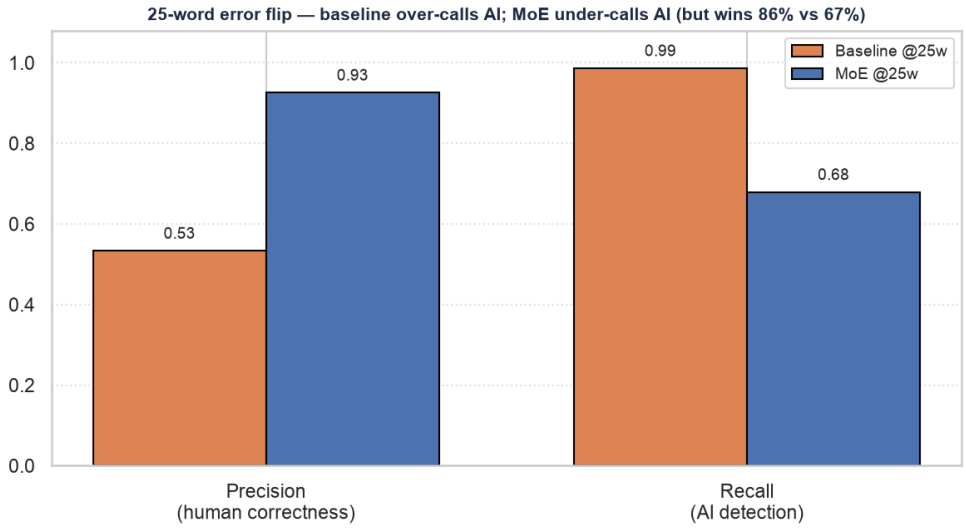


Figure 8: 25w precision/recall — accuracy hides the tradeoff.

7.2 Sentence ablation

1/2/3 sentences: baseline 59.8%/74.5%/83.4% → MoE 81.6%/90.6%/93.4%. Sentence truncations are harder than word truncations at matched budget (more padding loss), confirming sentence structure carries signal the short expert partially recovers via char features.

7.3 Type analysis

Type gate: essay easiest (long formal → distinct), chat next (short informal markers), casual hardest (middle). Routing insight: short+chat inputs have the lowest confidence and need warnings; short+essay (truncated) input needs the low threshold. Reporting type alongside label is a human-factor result, not just accuracy.

7.4 What did not help

Tuning long/verylong thresholds hurt full-text (98.96%→98.60% on eval) — small calib slices overfit. Global 0.60 (best for 100–150w essay truncations) hurt HC3 natural chat. Conclusion: calibrate only where the failure is large and monotonic (short), not everywhere.

8 Limitations and Future Work

- **Ultra-short floor.** Even MoE is 86% at 25w with wide uncertainty. Training saw ≥50w; adding 10–35w + few-shot chat would likely beat thresholds.
- **Boundary blending.** 150w essays prefer baseline (97.4% vs 95.6%); soft routing (weighted P across short/long) could interpolate.
- **Threshold transfer.** Essay-tuned θ hurts human-heavy chat (–0.6 pp). Per-type thresholds (essay vs chat) are the next step.
- **Generators.** HC3 is one 2023 ChatGPT snapshot; modern holdout is 5 models. Paraphrase/adversarial eval is missing.
- **Scale.** Full 290k training might add 0.1–0.3 pp; transformers (DistilBERT/RoBERTa) with chunking remain the Phase-2 ceiling test.

Ethics: P(AI) is a screening signal, not proof. Short-text warnings and type reporting are required UI, not optional captions.

9 Conclusion

Length is not a detail — it is the task when evaluating short text. One global TF-IDF model buys 99% on essays by learning to call short text AI. Specializing by length, grounding short experts in real chat data, labeling chat/casual/essay, and calibrating per length recovers +10 to +50 pp where it matters, with a 0.3 pp full-text tax. We release the full stack so the claim is checkable: `moe/download_small_text.py` → `train_moe.py` → `calibrate_thresholds.py` →

`predict_moe.py`.

References

- [1] Guo, B. et al. How Close is ChatGPT to Human Experts? arXiv:2301.07597 (2023); Hello-SimpleAI/HC3 (reddit_eli5/open_qa/finance).
- [2] Roy, R. et al. A Comprehensive Dataset for Human vs AI Generated Text Detection. arXiv:2510.22874 (2025) (Defactify).
- [3] Wang, B. et al. M4: Multi-Generator, Multi-Domain, Multi-Task, Multi-Label. arXiv:2310.13607 (2023).
- [4] Shazeer, N. et al. Outrageously Large Neural Networks: MoE. arXiv:1701.06538 (2017).
- [5] Hastie, T. et al. The Elements of Statistical Learning. Springer (2009).
- [6] Robertson, S. & Zaragoza, H. BM25 & TF-IDF. Found. Trends IR (2009).
- [7] Reimers, N. & Gurevych, A. Sentence-BERT. EMNLP (2019). Sanh, T. DistilBERT (2019); Liu, Y. RoBERTa (2019).
- [8] art nitolog/llm-generated-texts (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3, Command R+) — Hugging Face.
- [9] Mitchell, E. DetectGPT. arXiv:2301.11305 (2023) — zero-shot baseline family.

Appendices

A Hyperparameters

Component	Setting (all seeded)
Baseline word TF-IDF	(1,2) 25k, min_df 2, sublinear, clean_text_for_tfidf
Baseline char TF-IDF	char_wb (3,5) 50k, min_df 3, sublinear
Baseline classifiers	NB α=0.1; LR C=1.5 lbfgs 1000; LinearSVC C=1.0 + CalibratedCV(3)
Experts (each)	Word 15k + Char 25k $\rightarrow$ LR C=1.5 lbfgs 1000
Type gate	StandardScaler + LR C=1.0 lbfgs 1000 on 16 features

Table A1: Hyperparameters.

B Reproducibility checklist

- Datasplits: train subsample (10k/class, seed 42), 80/20 per pool (seed 42), calib 2.5k/eval 2.5k (seed 999), HC3/modern holdouts 15% (seed 42).
- Artifacts: `models/*.pkl` (baseline), `models/moe/expert_*.pkl`, `type_gate.pkl`, `thresholds.json`; metrics `results/moe_*.csv`.
- Indirection: truncation code is in `moe/type_features.py` (truncate_words/random_window); routing in `moe/train_moe.py::route_predict`.
- PDFs: `make_report.py` (6p), `moe/make_moe_appendix.py` (6p), `make_research_paper.py` (7p), this file (deep).

C Full routed table

Condition	Acc	Prec	Rec	F1	AUC
full	0.9896	0.9929	0.9790	0.9859	0.9990
first_25_words	0.8606	0.9259	0.6792	0.7836	0.9473
first_50_words	0.9312	0.9316	0.8794	0.9048	0.9796
first_100_words	0.9520	0.9140	0.9612	0.9370	0.9916
first_150_words	0.9558	0.9103	0.9774	0.9426	0.9946
first_200_words	0.9762	0.9662	0.9699	0.9680	0.9969
first_1_sent	0.8164	0.9066	0.5640	0.6954	0.9138
first_2_sent	0.9056	0.9288	0.8079	0.8641	0.9635
first_3_sent	0.9336	0.9253	0.8934	0.9091	0.9798
hc3_holdout	0.9852	0.9622	0.9906	0.9762	0.9990
modern_holdout	0.9570	0.9832	0.9649	0.9740	0.9891

Table C1: Routed MoE (results/moe_routed_metrics.csv).

Baseline counterpart in `results/moe_baseline_comparison.csv`; calibration detail in `results/moe_calibrated_metrics.csv`.

PART IV

Code Workflow & Run Guide

Reproducible pipeline: setup, training, inference, API, UI (7 pages).

7 pages · reproduced verbatim from the original document

Code Walkthrough & Run Guide

Classification of Human-Written and AI-Generated Text
National Institute of Technology Sikkim — CSE

Understand the workflow end-to-end • How to train, test, and serve the model in 6 ways
Dual TF-IDF (75k features) • Logistic Regression & Linear SVM — 99%+ test accuracy

Authors: **Pravesh Gedon Sharma (B240019)** & **Ankit Rawat (B320032)** • This PDF pairs with `Code_Workflow_and_Run_Guide.pdf` (this file) and `Project_Report_AI_Text_Classification.pdf` (results).

Contents

1 Overview & Architecture

2 Repository Map (file by file)

3 src/ — Shared Utilities

4 Training Workflow (train_baseline.py, 11 steps)

5 Inference Workflow (test/inference.py)

6 The 6 Ways to Run It (CLI, Folder, CSV, Terminal, API, LangChain, Streamlit)

7 How to Run — Copy-Paste Commands

8 Outputs & What Each File Means

9 Troubleshooting & FAQ

Appendix A: Key Snippets (load → vectorize → predict)

1 Overview & Architecture

The project turns an essay into a 75,000-dimensional sparse vector (25k word TF-IDF + 50k char TF-IDF) and asks a linear classifier: **Human or AI?**. Nothing else. That simplicity is the point — it trains in 33 seconds, runs on a laptop, exposes a REST API, a LangChain tool, a CLI, and a web UI, all from the same 4 `.pkl` files.

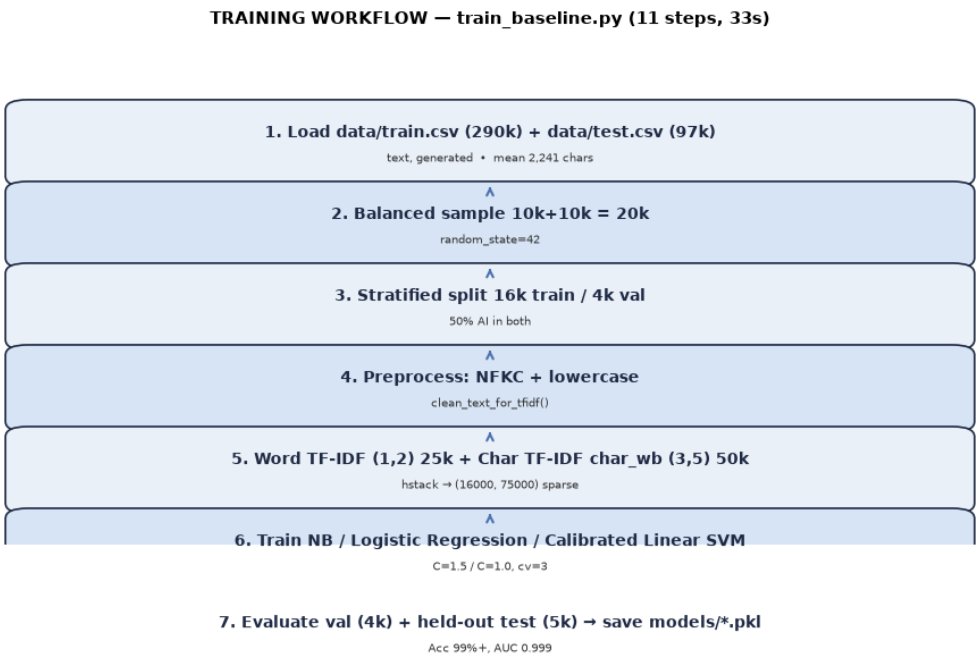


Figure 1: Training flow.

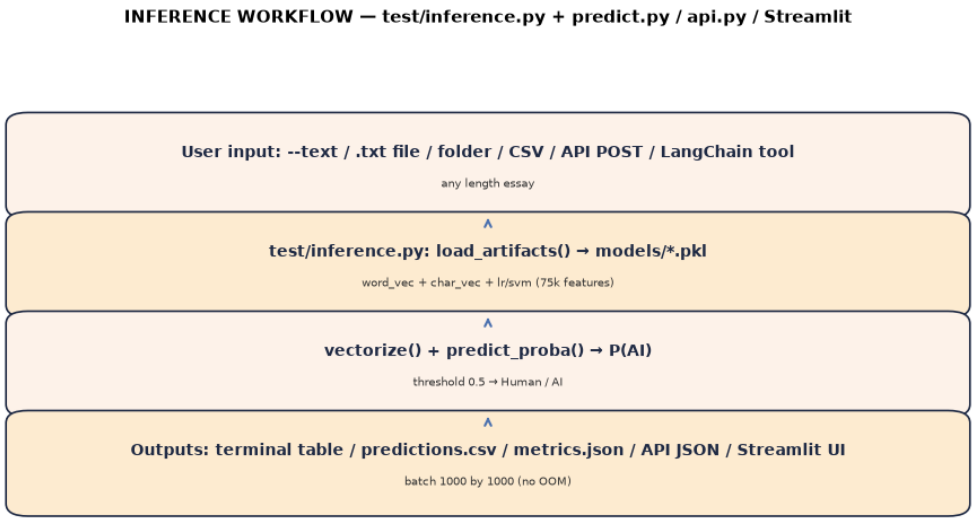


Figure 2: Inference flow (all 6 front-ends share test/inference.py).

- **Training (once):** `train_baseline.py` → `models/*.pkl` + `results/baseline_ml_metrics.csv`
- **Inference (every time):** `test/inference.py` loads the 4 pkls → `predict.py` / `api.py` / `langchain_tool.py` / `app/app.py` (Streamlit) → terminal answer + optional CSV/JSON.
- **Data:** `data/train.csv` 290,300 rows (balanced) / `data/test.csv` 97,447 rows. Columns: `text, generated` (0=Human, 1=AI).

2 Repository Map (file by file)

File	What it does	Lines
`train_baseline.py`	Full training pipeline (11 logged steps). Load → sample → split → TF-IDF → train NB/LR/SVM → evaluate → save.	142
`make_report.py`	Generates Project_Report_*.pdf (results + 5 figures) from live artifacts.	319
`make_run_guide.py`	Generates THIS guide PDF.	this
`src/preprocessing.py`	clean_text_for_tfidf() — NFKC + lowercase for TF-IDF.	45
`src/stylometrics.py`	TTR, burstiness, Flesch ease/grade, punctuation rates — used by Streamlit + tests.	162
`src/evaluate.py`	compute_classification_metrics(), plot_confusion_matrix(), plot_roc_curves().	95
`src/semantic.py`	Sentence-BERT embeddings + cosine — Phase 2 (not yet wired into inference).	80
`app/app.py`	Streamlit web UI (:8501). Loads models/ with @cache_resource, shows P(AI) + stylometrics.	110
`test/inference.py`	**Core inference engine.** load_artifacts(), vectorize(), predict_text/batch, evaluate_csv(), predict_folder().	158
`test/predict.py`	CLI: --text / --input / --folder / --csv / --interactive. Args → inference.py.	140
`test/api.py`	FastAPI (:8001): POST /predict, /predict/batch, /langchain/tool, GET /health, /model/info.	95
`test/langchain_tool.py`	LangChain BaseTool ai_human_classifier + batch tool. Agent-ready.	86
`test/terminal_demo.py`	Terminal demo comparing direct vs langchain vs API in one run (+ --loop chat).	81
`test/evaluate_examples.py`	Reads all test/example_texts/*.txt, predicts both models, prints aggregate metrics.	62
`test/test_model.py`	Smoke tests: single predict + 2k eval (assert acc>0.90). --full = 97k.	33

3 src/ — Shared Utilities

preprocessing.py

`clean_text_for_tfidf(text)` = NFKC → collapse whitespace → lowercase. Used as `preprocessor=` in word TF-IDF so train and test are normalised identically. `clean_text_for_stylometrics()` keeps casing/punctuation (capital ratio, commas) — never lowercases.

stylometrics.py

`extract_stylometric_features(text) → 24 floats` (num_words, ttr, hapax_ratio, mean_sentence_length, burstiness=σ/μ, comma_rate… flesch_reading_ease, flesch_kincaid_grade, gunning_fog). No sklearn — pure regex + syllable heuristics. Shown in Streamlit and returned by `--stylometrics` / API `stylometrics:true`.

evaluate.py / semantic.py

`evaluate.py` provides notebook plots (confusion + ROC) and metric dict. `semantic.py` (Sentence-BERT) is Phase-2 scaffolding — researched but **not** part of the 75k TF-IDF inference path.

4 Training Workflow (train_baseline.py, 11 steps, 33 s)

Run: `venv/bin/python train_baseline.py`. Each step prints a timestamped log line — watch them in order:

Step	What happens	Output
1	Load data/train.csv (290,300) + data/test.csv (97k metadata). Prints shape, class counts, length stats.	train (290300,2)
2	Balanced sample: per-class `sample(10000)` → concat → shuffle. Why 20k? Full 290k×75k TF-IDF is 14× slower; notebook 03 defines 20k.	(20000,2) 50/50
3	Stratified split (random_state=42, test_size=0.20) → 16k train / 4k val, both 50% AI.	train=16000 val=4000
4	Preprocessing demo: prints raw vs cleaned (lowercase) for 2 rows.	clean_text_for_tfidf
5	Word TF-IDF (1,2) 25k, min_df=2, sublinear_tf. fit on train, transform val.	(16000,25000) in 6.2s
6	Char TF-IDF char_wb (3,5) 50k, min_df=3. fit/transform.	(16000,50000) in 15.8s
7	Fusion hstack(word,char).tocsr(). Prints nnz (non-zero cells).	(16000,75000) nnz 34.8M
8	Train NB / LR (C=1.5) / LinearSVC+CalibratedCV(3). Each evals on 4k val: accuracy, precision, recall, F1, ROC-AUC, confusion, full report.	SVM 99.53% acc
9	Save results/baseline_ml_metrics.csv (Model,accuracy,…).	results/*.csv
10	Held-out test: load test.csv, sample 5000 (seed 42), transform with frozen vectorizers, LR+SVM eval.	LR 99.26% SVM 99.58%
11	Save 4 pkls to models/. Prints TOTAL TIME 33s.	models/*.pkl

Two pandas-3 fixes are baked in (your run hit them): per-class sampling instead of `groupby.apply` (which drops `generated`), and `np.array(..., dtype=object)` for Arrow-backed strings before sklearn split.

5 Inference Workflow (test/inference.py)

`test/inference.py` is the **one place** that touches models. Every front-end (CLI, API, LangChain, Streamlit) calls it — change inference once, all 6 ways update.

Function	One-liner
`load_artifacts()`	joblib.load ×4 from ../models/. Raises FileNotFoundError with 'Train first' hint if missing.
`vectorize(texts, art)`	word.transform + char.transform → hstack().tocsr(). Single str auto-wrapped to list.
`predict_proba(texts, art, model)`	clf.predict_proba(X)[:,1] → P(AI) array. model='lr'|'svm'.
`predict_text(text, art, model, stylometrics)`	Single-text dict: predicted_label (≥0.5→AI), probability_ai/human, confidence, optional 24 stylometrics.
`predict_batch(texts, batch_size=1000)`	Chunked to avoid OOM on 97k/290k CSVs. Returns np.array of P(AI).

Function	One-liner
`evaluate_csv(csv, sample_n)`	Loads CSV, batches predict, computes accuracy/precision/recall/F1/AUC + confusion + report, returns (metrics, df with prob_ai/pred/correct).
`predict_folder(folder)`	glib *.txt/*.md, predict_text each + stylometrics + file path.

6 The 6 Ways to Run It

#	Way	Command / Endpoint	Best for
1	CLI single / file	`test/predict.py --text "..." / --input file.txt`	One-off classify
2	CLI folder / CSV	`--folder test/example_texts` / `--csv data/test.csv --sample 5000`	Batch eval
3	Terminal chat	`test/terminal_demo.py --loop` or `predict.py --interactive`	Try text live
4	REST API	`test/api.py` → POST `/predict` (8001)	Other programs / curl
5	LangChain tool	`langchain_tool.ai_human_classifier.run(...)`	LLM agents
6	Web UI	`app/app.py` (Streamlit :8501)	Demo to humans

7 How to Run — Copy-Paste Commands (from project root)

A. First-time setup (once)

```
cd ~/Desktop/AI_generated_text_classification
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install "langchain-core>=0.3" "fastapi>=0.110" reportlab
# data: ensure data/train.csv (623MB) + data/test.csv (212MB) exist
ls -lh data/
```

B. Train (once — 33 seconds with data present)

```
venv/bin/python train_baseline.py
# → models/word_tfidf_vectorizer.pkl (969K)
#   models/char_tfidf_vectorizer.pkl (1.6M)
#   models/logistic_regression_baseline.pkl (587K)
#   models/linear_svm_baseline.pkl (1.7M)
#   results/baseline_ml_metrics.csv
```

C. Test / Inference — all 6 ways

```
# 1 — single text (terminal)
venv/bin/python test/predict.py --text "Your essay here..." --model lr --stylometrics

# 2 — file / folder (13 examples in test/example_texts/)
venv/bin/python test/predict.py --input test/example_texts/human_essay_school.txt --model svm
venv/bin/python test/predict.py --folder test/example_texts --model lr
venv/bin/python test/evaluate_examples.py          # custom evaluator (both models, aggregate)

# 3 — terminal chat (no browser)
venv/bin/python test/terminal_demo.py
venv/bin/python test/terminal_demo.py --api        # needs API running (see D)
venv/bin/python test/predict.py --interactive --model lr

# 4 — REST API (server, second terminal)
venv/bin/python test/api.py                        # → http://localhost:8001/docs
curl -X POST http://localhost:8001/predict -H "Content-Type: application/json" \
  -d '{"text":"In todays rapidly evolving digital landscape...","model":"lr","stylometrics":true}'

# 5 — LangChain tool (inside Python / agent)
venv/bin/python test/langchain_tool.py             # no-LLM demo
#   in agent: from test.langchain_tool import ai_human_classifier

# 6 — Web UI (second terminal)
venv/bin/streamlit run app/app.py --server.port 8501   # → http://localhost:8501
```

D. Full evaluation on whole files (batch inference)

```
venv/bin/python test/predict.py --csv data/test.csv --model lr --out test/results          # 97k rows
venv/bin/python test/predict.py --csv data/train.csv --model svm --out test/results_train_full # 290k rows
venv/bin/python test/test_model.py             # 2k-row smoke test
venv/bin/python test/test_model.py --full      # full 97k
```

E. Report PDF

```
venv/bin/python make_report.py        # → Project_Report_AI_Text_Classification.pdf (361KB)
venv/bin/python make_run_guide.py     # → Code_Workflow_and_Run_Guide.pdf (this file)
```

8 Outputs & What Each File Means

File	What it is
`models/*.pkl (4)`	Frozen vectorizers + classifiers. The ONLY things inference loads. Retrain to replace.
`results/baseline_ml_metrics.csv`	Val metrics per model (train_baseline.py step 9).
`test/results/predictions.csv`	Per-row: text, generated, prob_ai, pred, correct. Review errors here.
`test/results/metrics.json`	Same metrics as terminal + confusion matrix + full classification report.
`test/example_texts/`	13 .txt examples (human_/ai_ prefix = ground truth). `evaluate_examples.py` scores them.
`test/example_test_set.csv`	20-row balanced CSV for quick batch eval (--csv test/example_test_set.csv).
`results/figures/*.png`	5 figures for the PDF (class dist, length, comparison, confusion, ROC).

9 Troubleshooting & FAQ

- **FileNotFoundError: models/*.pkl missing** → you haven't trained yet. Run `venv/bin/python train_baseline.py` (needs `data/train.csv`).
- **KeyError 'generated' / Arrow TypeError** on pandas 3 → already fixed in train_baseline.py (per-class sampling + np.array dtype=object). Do not revert to groupby.apply.
- **API 'Connection refused' on 8001** → API not running. Start `venv/bin/python test/api.py` in a **separate terminal** (keep it open).
- **uvicorn test.api:app fails** → `test/` needs `__init__.py` (added) OR run `python test/api.py` directly (recommended).
- **Short text predicted wrong** (e.g. human_sample.txt 26 words → AI) → expected. Model is trained on ~400-word essays (mean 2,241 chars). Very short/clean text is out-of-distribution. Test with essay-length paths.
- **Streamlit not showing new model** → kill old process (`pkill -f "streamlit run"`) then restart — @cache_resource caches artifacts.
- **OOM on full train.csv** → don't load whole CSV in memory; use `--csv ... --sample 5000` or batch_size=1000 in `evaluate_csv()`.

Appendix A Key Snippets (load → vectorize → predict)

This is the entire inference in 10 lines — everything else is wrapper around it:

```
import joblib
from scipy.sparse import hstack

word_vec = joblib.load("models/word_tfidf_vectorizer.pkl")
char_vec = joblib.load("models/char_tfidf_vectorizer.pkl")
lr       = joblib.load("models/logistic_regression_baseline.pkl")

text = open("test/example_texts/ai_essay_distance_learning.txt").read()
X = hstack([word_vec.transform([text]), char_vec.transform([text])]).tocsr()
p_ai = lr.predict_proba(X)[0, 1]
print("AI-Generated" if p_ai >= 0.5 else "Human-Written", f"P(AI)={p_ai:.4f}")
# → AI-Generated P(AI)=0.9856
```

Higher-level: `from test.inference import load_artifacts, predict_text; r = predict_text(text, load_artifacts(), model='lr')`; REST: `POST /predict`; LangChain: `ai_human_classifier.run({'text': text})`.

Generated automatically by `make_run_guide.py` *from live artifacts.*